**INSTITUTO FEDERAL DE MATO GROSSO**

Tecnologia em Inteligência Artificial e Ciência de Dados · 4º semestre

# Aprendizagem Computacional

## Apostila do Aluno · versão 2

Do dado à decisão, uma pergunta de cada vez: teoria estruturada, exemplos e notebooks práticos

4 unidades · 24 capítulos · 3 notebooks · alinhada à ementa do PPC

**Prof. Arthur · Cuiabá-MT · 2026/2**

# Sumário


- Apresentação
- Unidade 1 — Tenho dados: o que eles dizem?
  - Cap. 1 — O que é aprender com dados
  - Cap. 2 — Vocabulário e paradigmas
  - Cap. 3 — O roteiro: CRISP-DM
  - Cap. 4 — Análise exploratória (EDA)
  - Prática da Unidade 1
- Unidade 2 — Meu primeiro modelo, provado com honestidade
  - Cap. 5 — O experimento honesto: treino, teste e baseline
  - Cap. 6 — Preparação dos dados e o vazamento
  - Cap. 7 — Distâncias e similaridade
  - Cap. 8 — Regressão
  - Cap. 9 — Classificação: regressão logística e métricas
  - Cap. 10 — Breve teoria da AC: por que aprender é possível
  - Prática da Unidade 2
- Unidade 3 — Outras famílias de modelos
  - Cap. 11 — Classificadores baseados em instâncias: k-NN
  - Cap. 12 — Modelos gráficos: Naive Bayes e redes bayesianas
  - Cap. 13 — Árvores de decisão
  - Cap. 14 — Máquinas de vetores de suporte (SVM)
  - Cap. 15 — Redes neuronais: do perceptron às redes para imagens
  - Cap. 16 — Combinação de classificadores (ensembles)
  - Prática da Unidade 3
- Unidade 4 — Sem y, e até a decisão
  - Cap. 17 — Redução de dimensionalidade: PCA
  - Cap. 18 — Análise de agrupamento: o que é um bom grupo
  - Cap. 19 — Agrupamento por partição: K-Means e K-Medoids
  - Cap. 20 — Agrupamento probabilístico: misturas gaussianas
  - Cap. 21 — Agrupamento difuso: Fuzzy C-Means
  - Cap. 22 — Mapas auto-organizáveis (SOM)
  - Cap. 23 — Detecção de anomalias
  - Cap. 24 — Aplicações: da previsão à decisão

  – Prática da Unidade 4
- Apêndices

# Apresentação

Esta apostila organiza a disciplina como uma **escada**: cada capítulo usa o anterior e prepara o próximo, e todos convergem para o mesmo lugar, o projeto da PPI. Se você já viu CRISP-DM, EDA e regressão, ótimo. Aqui eles reaparecem em ordem, ligados entre si e aplicados ao seu problema.

## Como usar esta apostila

**A pergunta que guia tudo.** Todo capítulo responde a uma parte desta frase:

> *Tenho um problema real e dados sobre ele. Como construo um modelo que eu possa provar que ajuda alguém a decidir melhor?*

- **Unidade 1** responde "tenho dados": o que é aprender, como nomear as peças e como conhecer os dados.
- **Unidade 2** responde "construo um modelo que posso provar": primeiro a régua, depois a preparação honesta, depois os primeiros modelos e a teoria que explica o que se viu.
- **Unidade 3** responde "existem outros jeitos de modelar?": as famílias de modelos e a comparação justa entre elas.
- **Unidade 4** responde "ajuda alguém a decidir": achar estrutura sem rótulo e levar qualquer resultado à decisão.

**A corrente de perguntas.** Nesta versão, cada capítulo responde a uma pergunta que o anterior deixou aberta, e termina com **A pergunta que fica**, que é a porta do capítulo seguinte. Se você ler só as "perguntas que ficam", em ordem, terá o roteiro da disciplina inteira.

**Como cada capítulo está montado**

1. **De onde viemos:** o que do capítulo anterior você precisa ter na cabeça.
2. **A ideia:** o conceito em linguagem simples, com uma analogia.
3. **Como funciona:** o mecanismo, com a matemática mínima necessária.
4. **No código:** o trecho de scikit-learn que faz aquilo.
5. **Armadilhas:** os erros mais comuns em projetos de turma.
6. **No seu projeto:** o passo da PPI que esse capítulo destrava.
7. **A pergunta que fica:** o que este capítulo ainda não resolve, e que o próximo vai responder.
8. **Verifique:** 3 perguntas para testar se entendeu, com resposta no fim.

**Os notebooks.** Os três notebooks (`AC_U1`, `AC_U2` e `AC_U3`) e o projeto-exemplo são os mesmos da primeira versão. Cada unidade desta apostila termina com uma **Prática** que diz qual seção de qual notebook corresponde a cada capítulo. Todos vêm em duas versões: `INICIAL` (com lacunas `TODO` para

você completar) e `COMPLETO` (gabarito comentado, que fica com o professor). Eles rodam no Google Colab sem configuração: *Arquivo → Enviar notebook*, depois *Ambiente de execução → Executar tudo*.

| Notebook de | Notebook | Dados |
|---|---|---|
| 1 | `AC_U1_Dados_ao_Pipeline` | Pinguins de Palmer (seaborn) |
| 2 | `AC_U2_Supervisionada` | Câncer de mama de Wisconsin + Diabetes (scikit-learn) |
| 3 | `AC_U3_Nao_Supervisionada` | Vinhos (scikit-learn) |
| PPI (cap. 24) | `PPI_Molde_Comparece_MT` | TSE, abstenção em MT: o projeto completo, do CSV ao dashboard |

**O repositório da disciplina.** Tudo isso está em `github.com/Schavess/ifmt-vg-ac`: a apostila, os notebooks `INICIAL` (cada um abre no Colab com o botão "Open in Colab" do README), as práticas semanais em `atividades/` e o projeto-exemplo da PPI em `ppi_molde_comparece_mt/`. Práticas extras semanais podem ser incluídas ao longo do semestre, sempre no mesmo formato: uma pasta por prática, com um `LEIAME`, um notebook com `TODO` e uma célula final de verificação que mostra ✅ ou ❌.

**Uma regra de ouro, repetida em todos os capítulos:** *problema → unidade de análise → y e X → separar treino e teste → baseline → modelo → métricas → interpretação.* Se você pular uma etapa, a conclusão não se sustenta.

## Por que esta ordem (e o que mudou da versão 1)

A primeira versão seguia a ordem da ementa do PPC, item por item. A ementa diz **o que** precisa ser estudado, mas não **em que ordem se aprende**. Esta versão cobre exatamente os mesmos 24 assuntos, com o mesmo texto, numa ordem em que cada capítulo depende do anterior. As principais mudanças:

- **O experimento honesto vem antes da preparação.** "Vazamento" só faz sentido para quem já sabe que existe um teste para proteger.
- **A teoria (PAC, VC, regularização) vem depois dos primeiros modelos**, quando você já viu o erro de treino mentir.
- **As famílias de modelos formam uma sequência**, cada uma respondendo a uma limitação da anterior, e terminam na comparação justa.
- **PCA e anomalias foram para a parte sem rótulo**, junto com o agrupamento, que usa as mesmas ideias.

| Cap. (v2) | Assunto | Cap. na v1 |
|---|---|---|
| 1 | O que é aprender com dados | 1 |
| 2 | Vocabulário e paradigmas | 2 |
| 3 | O roteiro: CRISP-DM | 3 |
| 4 | Análise exploratória (EDA) | 4 |
| 5 | O experimento honesto: treino, teste e baseline | 9 |
| 6 | Preparação dos dados e o vazamento | 5 |
| 7 | Distâncias e similaridade | 6 |
| 8 | Regressão | 11 |
| 9 | Classificação: regressão logística e métricas | 12 |
| 10 | Breve teoria da AC: por que aprender é possível | 10 |
| 11 | Classificadores baseados em instâncias: k-NN | 13 |
| 12 | Modelos gráficos: Naive Bayes e redes bayesianas | 14 |
| 13 | Árvores de decisão | 16 |
| 14 | Máquinas de vetores de suporte (SVM) | 17 |
| 15 | Redes neuronais: do perceptron às redes para imagens | 15 |
| 16 | Combinação de classificadores (ensembles) | 18 |
| 17 | Redução de | 7 |

| Cap. (v2) | Assunto | Cap. na v1 |
|---|---|---|
| | dimensionalidade: PCA | |
| 18 | Análise de agrupamento: o que é um bom grupo | 19 |
| 19 | Agrupamento por partição: K-Means e K-Medoids | 20 |
| 20 | Agrupamento probabilístico: misturas gaussianas | 21 |
| 21 | Agrupamento difuso: Fuzzy C-Means | 22 |
| 22 | Mapas auto-organizáveis (SOM) | 23 |
| 23 | Detecção de anomalias | 8 |
| 24 | Aplicações: da previsão à decisão | 24 |

## A ementa do PPC, capítulo a capítulo

A apostila segue a ementa oficial do componente (PPC do curso, 60 h): cada item da ementa tem pelo menos um capítulo. Use esta tabela para estudar para as avaliações.

| Item da ementa | Capítulo |
|---|---|
| Introdução. Conceitos e definições preliminares. Áreas de aplicação | 1 |
| Aprendizagem supervisionada, não supervisionada e por reforço. Exemplos de classificação, regressão e agrupamento | 2 |
| Tópicos de processamento de dados | 3 (CRISP-DM) e 6 (preparação e vazamento) |
| Medidas de dispersão e visualização de dados | 4 |
| Distâncias e similaridades | 7 |

| Item da ementa | Capítulo |
|---|---|
| Redução de dimensionalidade | 17 |
| Detecção de anomalias | 23 |
| Breve introdução à teoria da AC: aprendizagem PAC, dimensão VC e regularização | 10 (com 5: o experimento honesto) |
| Regressão | 8 |
| Classificação | 9 |
| Classificadores baseados em instâncias | 11 |
| Modelos gráficos | 12 |
| Redes neuronais (inclui redes para imagens e detecção, como o YOLO) | 15 |
| Árvores de decisão | 13 |
| Máquinas de vetores de suporte | 14 |
| Combinação de classificadores | 16 |
| Análise de agrupamento | 18 |
| Agrupamento por partição | 19 |
| Agrupamento probabilístico | 20 |
| Agrupamento difuso | 21 |
| Mapas auto-organizáveis | 22 |
| Aplicações | 24 e PPI GovTech |

**Ênfase tecnológica do PPC:** “abordagens para o desenvolvimento de classificação supervisionada e não supervisionada empregada na solução de problemas”. Por isso cada capítulo termina com **No seu projeto**: a teoria só está completa quando resolve um problema.

# Unidade 1 — Tenho dados: o que eles dizem?

A Unidade 1 responde à primeira parte da pergunta da apostila: **tenho um problema real e dados sobre ele**. O que é aprender com dados, como nomear cada peça, por onde começar e como conhecer os dados antes de mexer neles. Nenhum modelo ainda: só a base para não se perder depois.

## Cap. 1 — O que é aprender com dados

**De onde viemos:** nada. Este é o ponto de partida.

### A ideia

Um programa comum segue regras escritas por uma pessoa: "se a renda for menor que X, conceda o benefício". Em **Aprendizagem Computacional** (AC, *machine learning*), invertemos o caminho. Damos ao computador exemplos com as respostas, e ele **descobre a regra**. Depois usamos essa regra em casos novos.

*Analogia:* um corretor experiente estima o preço de um imóvel olhando área, bairro e número de quartos, porque já viu milhares de imóveis vendidos. Um modelo de AC faz o mesmo: "vê" muitos exemplos com o preço conhecido e aprende a relação entre o que é fácil observar (área, bairro) e o que se quer saber (preço).

### Três nomes que se confundem

| Termo | O que é | Pergunta típica |
|---|---|---|
| Inteligência Artificial (IA) | Campo amplo: sistemas que realizam tarefas que exigiriam inteligência humana | "Como fazer uma máquina decidir, conversar, enxergar?" |
| Ciência de Dados (CD) | Extrair conhecimento de dados para apoiar decisões, com estatística, programação e domínio | "O que os dados dizem sobre o problema?" |
| Aprendizagem Computacional (AC) | A parte da IA em que o sistema aprende padrões a partir de | "Que regra, aprendida dos dados, prevê bem casos novos?" |

| Termo | O que é | Pergunta típica |
|---|---|---|
| | exemplos | |

A AC está dentro da IA e é uma das ferramentas centrais da CD.

## Onde aparece e onde não deve aparecer

- **Funciona bem quando:** existem muitos exemplos históricos, o padrão se repete no futuro e errar é aceitável e mensurável. Exemplos: prever demanda de um posto de saúde, detectar veículos em câmeras de trânsito, triar denúncias.
- **Funciona mal quando:** há poucos dados, o mundo mudou (o passado não representa o futuro) ou a decisão exige explicação que o modelo não dá.
- **Riscos:** viés (o modelo reproduz injustiças presentes nos dados), privacidade (LGPD), excesso de confiança ("o sistema disse") e uso fora do contexto em que foi treinado.

## Armadilhas

- Achar que AC é "mágica" que dispensa entender o problema. O modelo só aprende o que está nos dados.
- Confundir **prever** com **explicar a causa**. Um modelo pode prever bem usando uma variável que não é causa de nada.

## No seu projeto

Escreva em uma frase: *que decisão pública o seu modelo vai apoiar, e quem toma essa decisão?* Se não houver decisão, ainda não há projeto de AC; há só uma análise.

**A pergunta que fica:** se a máquina aprende com exemplos, como chamamos cada peça desses exemplos, e que tipos de aprendizado existem?

## Verifique

1. Qual a diferença entre um sistema de regras e um modelo de AC?
2. Cite uma situação pública em que usar AC seria arriscado.
3. Prever bem significa ter descoberto a causa?

*Respostas:* (1) no sistema de regras a pessoa escreve a regra; na AC a regra é aprendida de exemplos. (2) Por exemplo, negar benefício social automaticamente com base em dados históricos enviesados. (3) Não: previsão usa associação, e causa exige outro tipo de evidência.

# Cap. 2 — Vocabulário e paradigmas

**De onde viemos:** a AC aprende uma regra a partir de exemplos. Agora vamos nomear as peças.

## O vocabulário mínimo

Pense numa planilha de imóveis à venda.

| Termo | Na planilha | Exemplo |
|---|---|---|
| **Exemplo** (amostra, instância) | Uma linha | Um imóvel anunciado |
| **Unidade de análise** | O que uma linha representa | Imóvel? Quarteirão? Bairro? Essa escolha muda tudo |
| **Atributos** (*features*, **X**) | Colunas de entrada | Área, bairro, quartos, idade do prédio |
| **Alvo** (*target*, rótulo, **y**) | A coluna que queremos prever | Preço (R$) |
| **Modelo** | A regra aprendida | "preço ≈ a × área + b × quartos" |
| **Treinar** (ajustar) | Achar os parâmetros da regra com exemplos conhecidos | Calcular *a* e *b* |
| **Prever** (inferir) | Aplicar a regra a casos novos | Estimar o preço de imóveis ainda não vendidos |

Um **identificador** (matrícula do imóvel, CPF, código do município) **nunca** é atributo. Ele não descreve o caso, só o nomeia. Se entrar em X, o modelo pode "decorar" casos em vez de aprender padrões.

## Os três paradigmas

- **Supervisionada:** os exemplos têm y conhecido. O modelo aprende X → y. São as Unidades 2 e 3.
- **Não supervisionada:** não há y. O modelo procura estrutura: grupos parecidos, direções de maior variação, casos estranhos. É a Unidade 4.
- **Por reforço:** não há tabela de exemplos com resposta. Um agente age num ambiente, observa o estado, recebe uma recompensa (ou punição) e aprende uma política: que ação tomar em cada estado para somar a maior recompensa ao longo do tempo. Exemplos: controlar o tempo de semáforos para reduzir filas, ajustar a irrigação conforme a umidade do solo, jogar xadrez. A ementa pede o conceito; os algoritmos (como o Q-learning) ficam para disciplinas futuras.

## Os tipos de problema

O **tipo do y** define a tarefa. A pergunta é: *o que eu quero como resposta?*

| Se a resposta é... | A tarefa é... | Exemplo público |
|---|---|---|
| Uma quantidade | **Regressão** | Quantas internações no próximo mês? Qual a abstenção do município? |
| Uma categoria | **Classificação** (binária ou multiclasse) | Esta obra vai atrasar (sim/não)? Que tipo de denúncia é esta? |
| Não há resposta: quero grupos | **Agrupamento** | Que perfis de município existem? |
| Não há resposta: quero o estranho | **Detecção de anomalias** | Que contratos fogem do padrão? |

A mesma situação pode virar tarefas diferentes. “Abstenção em %” é regressão; “abstenção acima de 25%? sim/não” é classificação. Escolha pela decisão que será tomada, não pelo algoritmo que você quer usar.

## No código

```
import seaborn as sns
pinguins = sns.load_dataset('penguins')   # 344 linhas: cada uma é um pinguim
y = pinguins['species']                   # alvo categórico → classificação
X = pinguins[['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm',
'body_mass_g']]
```

## Armadilhas

- Escolher a unidade de análise errada. “Aluno” e “escola” levam a modelos e conclusões diferentes.
- Achar que coluna de texto torna o problema classificação. Quem define é o y.

## No seu projeto

Preencha: *uma linha da minha base é ____; meu y é ____ (ou “não há y”); por isso minha tarefa é ____.*

**A pergunta que fica:** sabemos nomear X, y e a tarefa. Mas por onde se começa um projeto, e em que ordem?

## Verifique

1. Prever o preço de um imóvel é classificação ou regressão?
2. O código IBGE do município pode ser atributo?
3. Agrupar escolas por perfil é supervisionado?

*Respostas:* (1) regressão, porque y é uma quantidade. (2) Não, é identificador. (3) Não: não há y, é não supervisionado.

# Cap. 3 — O roteiro: CRISP-DM

**De onde viemos:** sabemos nomear X, y, a unidade de análise e a tarefa. Falta saber **em que ordem** fazer as coisas.

## A ideia

O **CRISP-DM** (*Cross-Industry Standard Process for Data Mining*) é um roteiro de 6 fases para projetos de dados. Ele é um **ciclo**, não uma linha reta: é normal voltar uma ou duas fases quando os dados mostram algo inesperado.

*Analogia:* é como uma reforma. Primeiro se define o objetivo (o que precisa mudar e para quem), depois se faz o levantamento do que existe, depois se prepara a obra, depois se executa e se acompanha. Se o levantamento revela um problema escondido, o plano é revisto.

## As 6 fases e o que cada uma entrega

| Fase | Pergunta central | Artefato que prova que a fase foi feita | Onde nesta apostila |
|---|---|---|---|
| 1. Entendimento do negócio | Que decisão queremos apoiar? Como saberemos que deu certo? | Frase do problema, decisor, pergunta, hipóteses, critério de sucesso | Caps. 1 e 2 |
| 2. Entendimento dos dados | Que dados existem? Eles respondem à pergunta? | Fontes com link, dicionário, EDA | Cap. 4 |
| 3. Preparação | Como transformar dado bruto em base analítica? | Script de ETL, `base_analitica.csv`, `Pipeline` | Caps. 6 e 7, 17 e 23 |
| 4. Modelagem | Que modelos testar, e contra qual baseline? | Tabela baseline × modelos | Caps. 5, 8–16 e 18–22 |
| 5. Avaliação | O modelo resolve o problema do negócio, não só a métrica? | Métricas no teste, análise de erros, limitações | Caps. 5 e 9 e 10 |
| 6. Implantação | Como o resultado | Dashboard, ranking, | Cap. 24 |

| Fase | Pergunta central | Artefato que prova que a fase foi feita | Onde nesta apostila |
|---|---|---|---|
| | chega a quem decide? | API, relatório | |

## ETL não é pré-processamento

Os dois nomes se confundem, mas acontecem em lugares diferentes.

- **ETL** (extrair, transformar, carregar) acontece **antes** do modelo e uma vez só. Junta fontes, corrige tipos, agrega na unidade de análise e salva a base analítica. É trabalho da disciplina de Extração e Transformação de Dados.
- **Pré-processamento** (imputar, escalar, codificar) faz parte **do modelo** e precisa ser aprendido só com o treino (Cap. 6). Ele mora dentro do `Pipeline`.

Uma regra prática: se a operação usa uma estatística calculada sobre os dados (média, desvio, categorias existentes), ela é pré-processamento e vai para o `Pipeline`. Se é uma regra fixa (converter texto em data, somar por município), pode ficar no ETL.

## Armadilhas

- Pular a fase 1 e começar pelo algoritmo. O resultado é um modelo que responde a uma pergunta que ninguém fez.
- Tratar o CRISP como formulário para preencher. Ele é um roteiro para **fazer**: cada fase precisa deixar um artefato.

## No seu projeto

O plano de sprints da PPI segue exatamente esta ordem. 03/11: fase 1. 10/11: fases 2–3. 17/11 e 24/11: fases 4–5. 01/12: fase 6.

**A pergunta que fica:** o roteiro manda entender os dados antes de modelar. O que, exatamente, se olha nos dados?

## Verifique

1. Por que o CRISP-DM é desenhado como ciclo?
2. Normalizar com a média da coluna é ETL ou pré-processamento?
3. Qual artefato prova que a fase de entendimento dos dados foi feita?

*Respostas:* (1) porque descobertas nos dados obrigam a rever o problema ou a preparação. (2) Pré-processamento: usa uma estatística dos dados e deve ser ajustado só no treino. (3) Fontes com link, dicionário e EDA.

# Cap. 4 — Análise exploratória (EDA)

**De onde viemos:** temos a pergunta e a base. Antes de modelar, precisamos **conhecer** os dados, como um médico que examina o paciente antes de receitar.

## A ideia

A **EDA** (*Exploratory Data Analysis*) é olhar os dados com resumos e gráficos para responder a quatro perguntas:

1. **Estrutura:** quantas linhas, que tipos de coluna, a unidade de análise está certa (sem duplicatas)?
2. **Qualidade:** há valores ausentes, impossíveis (idade negativa) ou inconsistentes?
3. **Distribuição:** como se comporta cada variável, e principalmente o y? Há assimetria, extremos, classes raras?
4. **Relações:** quais X parecem se relacionar com y? Há X redundantes entre si?

EDA **não é modelo**. Ela gera hipóteses e guia escolhas, mas não prova nada.

## Estatística descritiva que você vai usar

- **Média e mediana:** se forem muito diferentes, a distribuição é assimétrica. Renda e população costumam ser.
- **Desvio-padrão e quartis:** o quanto os valores se espalham. O intervalo interquartil (Q3 − Q1) é robusto a extremos.
- **Correlação de Pearson (r):** de −1 a +1, mede relação **linear**. r ≈ 0 não significa "sem relação": pode haver relação curva.
- **Contagem por classe:** em classificação, mostra se o problema é desbalanceado.

**Medidas de dispersão** (a ementa pede atenção a elas): dizem o quanto os valores se afastam do centro.

| Medida | Fórmula ou definição | Quando usar |
|---|---|---|
| Amplitude | máximo − mínimo | Visão rápida; muito sensível a um único extremo |
| Variância ($s^2$) | média dos quadrados dos desvios em relação à média (dividindo por n − 1) | Base do PCA e de muitos modelos; unidade ao quadrado |
| Desvio-padrão (s) | raiz da variância | Mesma unidade do dado; é o que o `StandardScaler` usa |
| Intervalo interquartil (IQR) | Q3 − Q1 | Robusto a extremos; define os |

| Medida | Fórmula ou definição | Quando usar |
|---|---|---|
| | | “bigodes” do boxplot e uma regra simples de anomalia (Cap. 23) |
| Coeficiente de variação (CV) | 100 × s ÷ média | Comparar dispersão de variáveis em unidades diferentes. Ex.: o tempo de espera numa UPA varia mais, proporcionalmente, que o número de atendimentos? |

$$s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2 \qquad CV = 100 \cdot \frac{s}{\bar{x}}$$

## Que gráfico para qual pergunta

| Pergunta | Gráfico |
|---|---|
| Como uma variável numérica se distribui? | Histograma ou boxplot |
| Como uma categórica se distribui? | Barras de contagem |
| Numérica × numérica | Dispersão (*scatter*) |
| Numérica por categoria | Boxplot por grupo |
| Muitas numéricas de uma vez | Mapa de calor de correlação ou `pairplot` |
| Evolução no tempo | Linha |

## No código

```
pinguins.info()                                  # tipos e não nulos
pinguins.describe()                              # resumo numérico
pinguins.isna().sum()                            # ausentes por coluna
pinguins['species'].value_counts(normalize=True) # balanceamento do y
sns.pairplot(pinguins, hue='species')            # relações, coloridas pelo y
pinguins.select_dtypes('number').corr()          # correlações
```

## Armadilhas

- **Correlação não é causalidade.** Municípios com mais bibliotecas têm menos evasão escolar? Ambos podem ser efeito da renda.

- **Falácia ecológica:** uma relação entre municípios não vale automaticamente para pessoas.
- **Olhar o teste na EDA e depois "escolher" variáveis por ele.** No projeto, decisões que dependem de y devem usar só o treino. A EDA geral (tipos, ausentes) pode usar a base toda.
- **Gráfico sem pergunta.** Dez gráficos bonitos que não respondem nada não são EDA.

### No seu projeto

Entrega mínima de 10/11: distribuição do y, tabela de ausentes por coluna e 3 gráficos de relação X × y, cada um com uma frase de leitura **sem** afirmar causa.

**A pergunta que fica:** conhecemos os dados e já temos vontade de treinar um modelo. Mas, antes de qualquer modelo: **como vamos saber se ele presta?**

### Verifique

1. Média de 18.579 e mediana de 7.788 eleitores por município: o que isso diz?
2. r = 0,02 entre duas variáveis prova que não há relação?
3. Para que serve contar as classes do y?

*Respostas:* (1) distribuição muito assimétrica: poucos municípios grandes puxam a média; vale usar log. (2) Não, pode haver relação não linear; olhe a dispersão. (3) Para saber se há classe rara, o que muda a métrica e a forma de separar treino e teste (Caps. 5 e 9).

## Prática da Unidade 1

| Onde | Capítulos | O que você vai ver |
|---|---|---|
| `AC_U1_Dados_ao_Pipeline`, seções 1–2 (estrutura, qualidade, EDA) | 2 e 4 | 2 aves sem medidas, 11 sem sexo; Adelie é 44% da base; nadadeira e massa com r = 0,87 |
| Prática de 27/10: O problema | 1 a 4 | Pergunta, unidade de análise, y, X e a ficha do problema da PPI |

### Checklist da Unidade 1 (vale para o seu projeto)

- ☐ Sei dizer a unidade de análise, o y e a tarefa em uma frase cada.
- ☐ Sei em que fase do CRISP-DM está cada entrega do meu grupo.
- ☐ Fiz EDA guiada por perguntas: estrutura, qualidade, distribuição, dispersão e relações.
- ☐ Listei cada coluna de X e confirmei que ela é conhecida antes do momento da decisão.

# Unidade 2 — Meu primeiro modelo, provado com honestidade

A Unidade 2 responde: **como construo um modelo que eu possa provar que funciona?** A ordem é proposital. Primeiro a régua (Cap. 5: separar o teste e criar o baseline), porque sem ela nenhum resultado vale. Depois a preparação honesta (Caps. 6 e 7), que só faz sentido quando já existe um teste para proteger. Então os dois primeiros modelos completos, regressão e logística (Caps. 8 e 9), e, por último, a teoria que explica o que você acabou de ver acontecer (Cap. 10).

## Cap. 5 — O experimento honesto: treino, teste e baseline

**De onde viemos:** a EDA (Cap. 4) mostrou como os dados são. Antes de tratar ausentes e escalas, e antes de escolher qualquer modelo, combinamos **como o modelo vai ser julgado**. É por isso que este capítulo vem primeiro: tudo o que vier depois será medido com esta régua.

### A ideia

O objetivo de um modelo é acertar casos **que ele nunca viu**. Medir no próprio treino é como um aluno corrigir a prova que ele mesmo escreveu, com o gabarito na mão. Por isso separamos:

| Conjunto | Para quê | Regra |
|---|---|---|
| **Treino** | Ajustar parâmetros e escolher modelos (com validação cruzada, cap. 10) | Pode ser usado à vontade |
| **Teste** | Estimar o desempenho em casos novos | Separado **primeiro**; usado **uma única vez**, no fim |

*Analogia:* um simulado só mede o preparo do aluno se as questões forem novas. Se forem as mesmas da lista de exercícios, a nota será otimista.

### Como separar

- **Aleatório** (`train_test_split`, 20% a 30% para teste): o padrão quando as linhas são independentes.
- **Estratificado** (`stratify=y`): em classificação, mantém a proporção das classes nos dois lados. Indispensável com classe rara.

- **Temporal:** se o modelo vai prever o futuro, treine no passado e teste no período mais recente. Sorteio aleatório em série temporal é vazamento do futuro.
- **Por grupo** (`GroupShuffleSplit`): se há várias linhas do mesmo município ou paciente, todas devem ficar do mesmo lado.

## O baseline: a régua mínima

O **baseline** é o modelo mais burro possível: na regressão, prever sempre a média; na classificação, chutar sempre a classe mais frequente (`DummyRegressor`, `DummyClassifier`, ZeroR no Weka). Ele responde a uma pergunta que quase todo trabalho esquece: *o seu modelo é melhor que não ter modelo?*

Nos dados de câncer de mama do notebook, o baseline acerta **62,7%** dos casos sem olhar nenhuma medida, só porque 62,7% são benignos. Uma acurácia de 70% parece boa, mas é pouco melhor que nada.

## O fluxo completo

*problema → unidade de análise → y e X →* ***separar*** *→* ***baseline*** *→ modelo em* `Pipeline` *→ métricas → interpretação*

```
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, stratify=y,
random_state=42)
baseline = DummyClassifier(strategy='most_frequent').fit(X_tr, y_tr)
modelo = make_pipeline(StandardScaler(), LogisticRegression()).fit(X_tr, y_tr)
print(baseline.score(X_te, y_te), modelo.score(X_te, y_te))
```

## Armadilhas

- Ajustar o modelo, olhar o teste, mudar algo, olhar o teste de novo. A cada olhada, o teste vira treino.
- Reportar só o modelo, sem baseline.
- Esquecer o `random_state`: ninguém consegue reproduzir seu número.

## No seu projeto

Entrega de 17/11: tabela com **baseline e pelo menos um modelo**, com o mesmo split e a mesma métrica.

**A pergunta que fica:** separamos o teste. Mas a EDA mostrou ausentes, texto e escalas diferentes. Como preparar os dados **sem deixar o teste vazar** para o treino?

## Verifique

1. Por que o teste só pode ser usado uma vez?
2. Quando o split aleatório é errado?

3. Para que serve o baseline?

*Respostas:* (1) porque decisões tomadas depois de vê-lo o transformam em dado de treino, e a estimativa fica otimista. (2) Em séries temporais e quando há várias linhas do mesmo grupo. (3) Para mostrar quanto o modelo agrega em relação a uma regra trivial.

# Cap. 6 — Preparação dos dados e o vazamento

**De onde viemos:** o Cap. 5 separou treino e teste e criou o baseline. Agora a palavra "vazamento" faz sentido: é qualquer informação do teste entrando no treino. A EDA mostrou ausentes, colunas em texto e escalas diferentes; vamos tratar isso **sem trapacear**.

## Os três tratamentos clássicos

| Problema | Tratamento | Classe do scikit-learn | Detalhe |
|---|---|---|---|
| Valores ausentes | **Imputação**: preencher com mediana (numéricas) ou valor mais frequente (categóricas) | `SimpleImputer` | Se a ausência em si for informativa, crie uma coluna "estava ausente" (`add_indicator=True`) |
| Colunas categóricas | **Codificação** *one-hot*: uma coluna 0/1 por categoria | `OneHotEncoder(handle_unknown='ignore')` | Não use 1, 2, 3 para categorias sem ordem: o modelo acharia que 3 é "maior" que 1 |
| Escalas diferentes | **Padronização**: (valor − média) ÷ desvio | `StandardScaler` | Essencial para k-NN, SVM, MLP e regressão com regularização; árvores não precisam |

## O vazamento de dados (*data leakage*)

**Vazamento** é quando informação que o modelo não teria no momento real de uso entra no treinamento. O resultado é sempre o mesmo: métricas ótimas no notebook e fracasso no mundo real. Ele aparece de três formas.

1. **Vazamento do alvo:** uma coluna de X é consequência de y ou faz parte do cálculo dele. Exemplo: prever se um paciente terá alta usando "data da alta". Ou prever abstenção usando o número de votos do mesmo turno.

2. **Vazamento do futuro:** usar informação posterior ao momento da previsão. Exemplo: prever os acidentes de trânsito de 2025 com uma variável medida em 2026.
3. **Vazamento do teste:** calcular média, escala ou categorias com a base inteira **antes** de separar treino e teste. O modelo "espia" o teste.

*Analogia:* é como o professor entregar a prova com o gabarito junto para os alunos "estudarem". A nota vai ser ótima, mas não mede o que eles aprenderam.

### A solução: `Pipeline` e `ColumnTransformer`

Um `Pipeline` encadeia o pré-processamento e o modelo num objeto só. Quando você chama `fit` no treino, **cada etapa aprende só com o treino**. Quando chama `predict` no teste, as etapas **aplicam** o que aprenderam, sem reaprender. O `ColumnTransformer` aplica tratamentos diferentes a colunas diferentes.

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.neighbors import KNeighborsClassifier

num = ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g']
cat = ['island', 'sex']

preparo = ColumnTransformer([
    ('num', Pipeline([('imputa', SimpleImputer(strategy='median')),
                      ('escala', StandardScaler())]), num),
    ('cat', Pipeline([('imputa', SimpleImputer(strategy='most_frequent')),
                      ('codifica', OneHotEncoder(handle_unknown='ignore'))]),
cat),
])
modelo = Pipeline([('preparo', preparo), ('knn',
KNeighborsClassifier(n_neighbors=5))])

modelo.fit(X_treino, y_treino)        # tudo aprende só com o treino
print(modelo.score(X_teste, y_teste)) # o teste só é transformado, nunca
aprendido
```

O `Pipeline` também resolve o vazamento na validação cruzada (Cap. 10), porque o pré-processamento é reajustado em cada dobra automaticamente.

### Armadilhas

- `StandardScaler().fit_transform(X)` antes do `train_test_split`. É o erro número 1 em trabalhos de turma.
- Remover linhas com ausentes sem verificar se elas são diferentes das outras. Isso pode criar viés.
- Usar `LabelEncoder` em X. Ele é feito para y.

### No seu projeto

Toda transformação que usa estatística dos dados vai para dentro de um `Pipeline`. Liste as colunas de X e marque, para cada uma: "eu saberia este valor **antes** do momento da decisão?". Se a resposta for não, a coluna sai.

**A pergunta que fica:** padronizamos as colunas porque "muitos métodos medem distância". Distância de quê, e por que a escala decide tudo?

### Verifique

1. Por que o `StandardScaler` deve ser ajustado só no treino?
2. Prever inadimplência usando "número de cobranças enviadas": há problema?
3. Por que não codificar "Norte, Sul, Leste" como 1, 2, 3?

*Respostas:* (1) porque a média e o desvio do teste seriam informação "do futuro" e inflariam o resultado. (2) Sim: as cobranças acontecem por causa da inadimplência, é vazamento do alvo. (3) Porque o modelo interpretaria uma ordem e distâncias que não existem; use *one-hot*.

## Cap. 7 — Distâncias e similaridade

**De onde viemos:** sabemos padronizar colunas dentro de um `Pipeline` (Cap. 6). Agora entendemos **por que** isso importa tanto: muitos métodos funcionam medindo o quanto dois exemplos são parecidos.

### A ideia

Se cada exemplo é uma linha de números, ele é um **ponto num espaço**. Um pinguim com bico de 40 mm e massa de 4.000 g é o ponto (40, 4000). "Parecido" passa a significar **perto**. O k-NN (Cap. 11), o K-Means (Cap. 19) e vários detectores de anomalia (Cap. 23) dependem disso.

### As medidas mais usadas

Para dois pontos a e b com p colunas:

$$\text{Euclidiana}: \; d(a,b) = \sqrt{\sum_{j=1}^{p} (a_j - b_j)^2} \qquad \text{Manhattan}: \; d(a,b) = \sum_{j=1}^{p} |a_j - b_j|$$

- **Euclidiana:** a distância "em linha reta". É o padrão.
- **Manhattan:** a soma das diferenças, como andar por quarteirões. Menos sensível a uma coluna com diferença enorme.
- **Cosseno:** compara a **direção** dos vetores, não o tamanho. É usada com textos (dois documentos sobre o mesmo tema, um longo e outro curto, são parecidos).

## Por que a escala decide tudo

Compare dois pinguins: diferença de 5 mm no bico e de 300 g na massa. Na distância euclidiana, a massa contribui com $300^2 = 90.000$ e o bico com $5^2 = 25$. O bico praticamente **desaparece**, só porque foi medido em milímetros e a massa em gramas. Depois de padronizar, as duas colunas passam a falar "em desvios-padrão" e contribuem de forma comparável.

*Outro exemplo:* comparar municípios por "população" (milhares) e "taxa de alfabetização" (0 a 100) sem padronizar faz a população mandar sozinha na similaridade.

## No código

```
from sklearn.metrics import pairwise_distances
from sklearn.preprocessing import StandardScaler

num = pinguins[['bill_length_mm', 'body_mass_g']].dropna()
bruto = pairwise_distances(num.iloc[:3])                                    #
dominado pela massa
padronizado = pairwise_distances(StandardScaler().fit_transform(num)[:3])    #
equilibrado
```

## A maldição da dimensionalidade

Com muitas colunas, todos os pontos tendem a ficar "igualmente longe" uns dos outros, e a noção de vizinho perde sentido. É um motivo para escolher bem as variáveis e, às vezes, reduzir dimensão com PCA (Cap. 17).

## Armadilhas

- Usar k-NN ou K-Means sem padronizar.
- Incluir uma coluna *one-hot* com dezenas de categorias e deixar que ela domine a distância.

## No seu projeto

Se você pretende agrupar municípios, escolas ou unidades de saúde, decida **quais colunas definem "parecido"** para o gestor. Essa é uma decisão de negócio, não só técnica.

**A pergunta que fica:** temos protocolo (Cap. 5) e preparação honesta (Caps. 6 e 7). Está na hora do primeiro modelo completo. Qual é o mais simples que resolve um problema de verdade?

## Verifique

1. Por que padronizar antes de calcular distâncias?
2. Quando a distância do cosseno é preferível?
3. O que é a maldição da dimensionalidade?

*Respostas:* (1) para que colunas com unidades grandes não dominem a distância. (2) Quando importa a proporção entre as colunas e não o tamanho, como em textos. (3) Com muitas colunas, as distâncias ficam parecidas entre todos os pares e o "vizinho mais próximo" perde significado.

# Cap. 8 — Regressão

**De onde viemos:** temos o protocolo (Cap. 5), a preparação dentro do `Pipeline` (Cap. 6) e a escala (Cap. 7). A regressão é o primeiro modelo completo: y é uma **quantidade**.

## A ideia: regressão linear

A regressão linear supõe que y é aproximadamente uma soma ponderada das colunas de X:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ \ldots \ + \beta_p x_p$$

O ajuste escolhe os coeficientes β que **minimizam a soma dos quadrados dos resíduos** (mínimos quadrados), sendo o resíduo a diferença real − previsto. Cada $\beta_j$ diz quanto ŷ muda quando $x_j$ aumenta uma unidade, **mantidas as outras constantes**.

*Analogia:* é a mesma reta de tendência que você já traçou numa planilha. A diferença é o foco: aqui não basta "ajustar bem" os dados que se tem; o que importa é "prever bem casos novos".

## Ridge e Lasso: a regressão com freio

Com muitas colunas, ou colunas correlacionadas, os β ficam instáveis (variância alta, cap. 10). A regularização soma uma penalidade ao erro:

| Modelo | Minimiza | Efeito |
|---|---|---|
| Linear (OLS) | soma dos resíduos² | Sem freio |
| **Ridge (L2)** | resíduos² + α · Σ $\beta_j^2$ | Encolhe todos os coeficientes; lida bem com colunas correlacionadas |
| **Lasso (L1)** | resíduos² + α · Σ $\lvert\beta_j\rvert$ | Zera coeficientes pouco úteis: faz seleção de variáveis |
| Elastic Net | mistura das duas | Meio-termo |

O α é escolhido por validação cruzada (`RidgeCV`, `LassoCV`). Como a penalidade depende do tamanho dos β, **padronize X antes** (dentro do `Pipeline`).

## Métricas de regressão

| Métrica | O que mede | Leitura |
|---|---|---|
| **MAE** (erro absoluto médio) | Média de \|real − previsto\| | "Erra, em média, X unidades". Robusto a extremos; fácil de explicar ao gestor |
| **RMSE** (raiz do erro quadrático médio) | Raiz da média dos erros² | Mesma unidade de y; pune mais os erros grandes |
| **$R^2$** (coeficiente de determinação) | 1 − erro do modelo ÷ erro de prever a média | 1 = perfeito; 0 = igual ao baseline; negativo = pior que o baseline |
| **MAPE** | Média de \|erro\| ÷ \|real\| | Em %, mas explode quando y é perto de zero |

O $R^2$ já embute a comparação com o baseline da média. Mesmo assim, reporte também o MAE, que tem unidade e significado para quem decide.

## No código (notebook da unidade, Parte A)

```
regressores = {
    'Baseline': DummyRegressor(),
    'Linear': make_pipeline(StandardScaler(), LinearRegression()),
    'Ridge': make_pipeline(StandardScaler(), Ridge(alpha=10)),
    'Lasso': make_pipeline(StandardScaler(), LassoCV(cv=5)),
}
```

Nos dados de diabetes (442 pacientes, 10 variáveis), a validação cruzada deu MAE de 66,9 para o baseline e cerca de 45 para os três modelos lineares, com $R^2$ de 0,48. O Lasso zerou 3 variáveis (idade, s2 e s4) sem perder desempenho. No teste, a linear teve MAE de 42,8 contra 64,0 do baseline.

## Diagnóstico pelos resíduos

Faça o gráfico resíduo × previsto. Uma nuvem sem padrão em torno de zero é o desejado. Um **funil** indica erro que cresce com o valor (teste log(y)). Uma **curva** indica relação não linear (inclua termos ao quadrado ou use árvores, cap. 13).

## Armadilhas

- Interpretar coeficientes como causa. "IMC tem o maior coeficiente" significa associação, mantidas as demais, nos dados observados.
- Comparar o tamanho de β entre colunas não padronizadas.
- Usar regressão linear para y que só pode ser 0 ou 1. Isso é classificação (cap. 9).

### No seu projeto

Se seu y é quantidade (abstenção, internações, custo de obra), comece por linear e Ridge contra o `DummyRegressor`, reportando MAE e $R^2$ em validação cruzada. É exatamente o que o PPI-molde faz.

**A pergunta que fica:** a regressão prevê quantidades. E quando o y é uma categoria (sim ou não, atrasa ou não atrasa)?

### Verifique

1. O que significa um $R^2$ negativo?
2. Por que padronizar antes do Ridge?
3. Qual a vantagem do Lasso sobre o Ridge?

*Respostas:* (1) Que o modelo é pior do que prever sempre a média. (2) Porque a penalidade compara o tamanho dos coeficientes, que depende da unidade de cada coluna. (3) Ele zera coeficientes e, assim, seleciona variáveis.

## Cap. 9 — Classificação: regressão logística e métricas

**De onde viemos:** a regressão (Cap. 8) prevê quantidades. Quando y é uma **categoria** (maligno/benigno, obra atrasa/não atrasa), precisamos de outro tipo de saída e, principalmente, de outras métricas.

### A ideia: regressão logística

Apesar do nome, é um **classificador**. Ela calcula uma soma ponderada, como a linear, e passa o resultado pela função **sigmoide**, que transforma qualquer número numa **probabilidade** entre 0 e 1:

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \ \ldots\ + \beta_p x_p)}}$$

Depois, um **limiar** (por padrão 0,5) transforma a probabilidade em classe. A fronteira entre as classes é linear. A regularização vem embutida: o parâmetro `C` é o inverso da força do freio (C menor = mais freio). A versão multiclasse funciona igual, com uma probabilidade por classe (softmax).

### A matriz de confusão

Tudo começa por definir a **classe positiva**: aquela que queremos encontrar (maligno, fraude, obra atrasada). Então:

| | Previsto positivo | Previsto negativo |
|---|---|---|
| **Real positivo** | VP (verdadeiro positivo) | FN (falso negativo): **o caso que escapou** |
| **Real negativo** | FP (falso positivo): **o alarme** | VN (verdadeiro negativo) |

| | Previsto positivo | Previsto negativo |
|---|---|---|
| | **falso** | |

## As métricas e quando cada uma importa

| Métrica | Fórmula | Pergunta que responde |
|---|---|---|
| Acurácia | (VP + VN) ÷ total | Quantos acertos no geral? **Enganosa com classes desbalanceadas** |
| **Precisão** | VP ÷ (VP + FP) | Dos que apontei como positivos, quantos eram? (custo do alarme falso) |
| **Recall** (sensibilidade) | VP ÷ (VP + FN) | Dos positivos reais, quantos encontrei? (custo do caso perdido) |
| **F1** | média harmônica de precisão e recall | Equilíbrio entre as duas |
| ROC-AUC | área sob a curva TPR × FPR | Ordena bem positivos acima de negativos? Independe do limiar |
| **PR-AUC** (*average precision*) | área sob a curva precisão × recall | Como a ROC, mas mais honesta quando o positivo é raro |

**Regra prática:** se perder um positivo é grave (doença, fraude, risco), priorize recall. Se cada alarme gera um custo alto (uma fiscalização cara), priorize precisão. Se o positivo é raro, nunca reporte só a acurácia.

## O limiar é uma decisão de negócio

No notebook, a logística no câncer de mama (1 = maligno), com previsões fora da dobra no treino, deu:

| Limiar | Recall | Precisão | Malignos perdidos (FN) | Alarmes falsos (FP) |
|---|---|---|---|---|
| 0,5 | 0,943 | 0,980 | 9 | 3 |
| 0,3 | 0,975 | 0,951 | 4 | 8 |
| 0,1 | 0,975 | 0,861 | 4 | 25 |

Baixar o limiar para 0,3 salva 5 casos ao custo de 5 alarmes falsos a mais. Baixar para 0,1 não salva mais nenhum e triplica os alarmes. Quem escolhe o ponto é o custo de cada erro e a capacidade do serviço de verificar os alertas, não o algoritmo.

## No código

```
logit = make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000))
proba = cross_val_predict(logit, X_tr, y_tr, cv=cv, method='predict_proba')[:,
1]
print(classification_report(y_tr, (proba >= 0.5).astype(int)))
print(roc_auc_score(y_tr, proba), average_precision_score(y_tr, proba))
```

## Classes desbalanceadas

Com 2% de positivos, um modelo que diz sempre "negativo" tem 98% de acurácia e recall zero. Remédios: métricas certas (recall, F1, PR-AUC), `stratify` no split, `class_weight='balanced'` nos modelos e ajuste do limiar.

## Armadilhas

- Esquecer de dizer qual é a classe positiva. Precisão e recall mudam completamente.
- Escolher o limiar olhando o teste. Use previsões fora da dobra no treino, como acima.
- Tratar a probabilidade como certeza: 0,51 e 0,99 viram a mesma classe, mas não significam a mesma coisa.

## No seu projeto

Escreva: *minha classe positiva é ___; perder um positivo custa ___; um alarme falso custa ___; por isso minha métrica principal é ___.*

**A pergunta que fica:** nos Caps. 8 e 9, o erro no treino foi sempre menor que no teste, e o Ridge e o C "seguraram" o modelo. Por que o treino engana, e quanto dado é preciso para confiar num modelo?

## Verifique

1. Por que a acurácia engana com classes desbalanceadas?
2. Num programa de fiscalização com equipe pequena, qual métrica priorizar?
3. O que muda ao baixar o limiar?

*Respostas:* (1) Porque chutar sempre a classe majoritária já dá acurácia alta. (2) Precisão, porque cada alarme consome uma visita cara. (3) Sobe o recall e cai a precisão: menos casos escapam, mais alarmes falsos.

# Cap. 10 — Breve teoria da AC: por que aprender é possível

**De onde viemos:** nos Caps. 8 e 9 você já viu o erro de treino mentir e a regularização (Ridge, Lasso, o C da logística) ajudar. A teoria agora explica **por que** o treino engana e **quanto** dado é preciso para confiar num modelo. A ementa pede três ideias: aprendizagem PAC, dimensão VC e sua relação com regularização.

## Erro de treino × erro de generalização

- **Erro de treino (empírico):** quanto o modelo erra nos exemplos que viu.
- **Erro de generalização (verdadeiro):** quanto erra em casos novos da mesma população. É o que importa, e só pode ser **estimado** (pelo teste ou pela validação cruzada).

A diferença entre os dois cresce quando o modelo é muito flexível para a quantidade de dados. Isso é o **sobreajuste** (*overfitting*): o modelo decora o ruído. O oposto, um modelo simples demais que erra até no treino, é o **subajuste** (*underfitting*).

*Analogia:* um aluno que decora as 20 questões da lista acerta todas elas (erro de treino zero), mas vai mal na prova com questões novas. Um aluno que só sabe uma regra genérica erra até a lista (subajuste). O bom aprendizado está no meio: entender o padrão, não decorar os casos.

## Viés e variância

| | Viés alto | Variância alta |
|---|---|---|
| O que é | Erra de forma sistemática, porque não consegue representar o padrão | Muda muito quando os dados de treino mudam um pouco |
| Sintoma | Erro alto no treino **e** na validação | Erro baixo no treino, alto na validação |
| Exemplo | Regressão linear para uma relação curva | Árvore sem limite de profundidade |
| Remédio | Modelo mais flexível, mais variáveis | Regularização, mais dados, combinação de modelos (cap. 16) |

## Aprendizagem PAC

A teoria **PAC** (*Probably Approximately Correct*, Valiant, 1984) formaliza o que é "aprender". Um algoritmo aprende no sentido PAC se, com exemplos suficientes, ele entrega com **alta probabilidade** (pelo menos 1 − δ) um modelo **aproximadamente correto** (erro de generalização no máximo ε).

Para um conjunto **finito** H de modelos possíveis, e supondo que um deles acerta tudo, basta um número de exemplos m que satisfaça:

$$m \geq \frac{1}{\varepsilon}\left(\ln|H| + \ln\frac{1}{\delta}\right)$$

A leitura importa mais que a conta. Quanto **mais modelos possíveis** (|H| maior, isto é, um modelo mais flexível), **mais dados** são necessários. Quanto **mais precisão** (ε menor) e **mais confiança** (δ menor) você exige, mais dados também.

## Dimensão VC

A maioria dos modelos tem infinitas configurações (uma reta pode ter qualquer inclinação), então |H| é infinito. A **dimensão de Vapnik–Chervonenkis (VC)** mede a capacidade do modelo de outra forma: é o **maior número de pontos que o modelo consegue separar de todas as maneiras possíveis** ("estilhaçar", *shatter*).

- Uma reta no plano separa 3 pontos (não alinhados) em qualquer combinação de rótulos, mas não 4 (pense no padrão de um "X"). Logo, VC = 3. Em p dimensões, um classificador linear tem VC = p + 1.
- Uma árvore sem limite de profundidade ou um k-NN com k = 1 têm capacidade enorme: decoram qualquer rotulagem.

Os limites de generalização de Vapnik dizem, de forma simplificada:

$$\text{erro verdadeiro} \;\leq\; \text{erro de treino} + \text{termo que cresce com } \sqrt{\frac{VC}{m}}$$

Com m (exemplos) fixo, um modelo de VC maior pode ter erro de treino menor, mas a **margem de segurança** cresce. É o sobreajuste em linguagem matemática.

## A ponte com a regularização

Se a capacidade excessiva causa sobreajuste, a solução é **limitar a capacidade**. Esse é o princípio da **minimização do risco estrutural** (Vapnik): escolher o modelo que equilibra erro de treino e complexidade. Na prática, cada família tem seu "freio":

| Modelo | Freio (hiperparâmetro de regularização) | Cap. |
|---|---|---|
| Regressão linear e logística | Ridge (L2), Lasso (L1): penalizam coeficientes grandes; `alpha`, `C` | 11, 12 |
| k-NN | k maior suaviza a fronteira | 13 |
| Rede neural | `alpha` (L2), menos neurônios, | 15 |

| Modelo | Freio (hiperparâmetro de regularização) | Cap. |
|---|---|---|
| | parada antecipada | |
| Árvore | `max_depth`, `min_samples_leaf`, poda | 16 |
| SVM | `C` menor = margem mais larga = menor capacidade efetiva | 17 |

O SVM é o exemplo mais direto da teoria: maximizar a margem é uma forma de controlar a capacidade, mesmo em muitas dimensões.

## Da teoria à prática: validação cruzada e busca de hiperparâmetros

Os limites teóricos são folgados demais para escolher o `max_depth` do seu modelo. Na prática, estimamos o erro de generalização **empiricamente**:

1. **Validação cruzada k-fold:** divide o **treino** em k partes, treina k vezes deixando uma parte de fora a cada vez, e reporta a **média e o desvio** das k medidas. Em classificação, use `StratifiedKFold`.
2. `GridSearchCV`**:** repete a validação cruzada para cada combinação de hiperparâmetros e escolhe a melhor.
3. **Teste:** só depois de tudo escolhido, uma vez.

```
busca = GridSearchCV(DecisionTreeClassifier(random_state=42),
                     {'max_depth': [2, 3, 4, 5, 6, None]},
                     cv=StratifiedKFold(5, shuffle=True, random_state=42),
scoring='f1')
busca.fit(X_tr, y_tr)
print(busca.best_params_, busca.best_score_)
```

No notebook da unidade, a curva de complexidade da árvore mostra a teoria acontecendo: o F1 no treino sobe até 1,0 conforme a profundidade cresce, e o F1 na validação para de melhorar por volta da profundidade 5.

## Armadilhas

- Escolher hiperparâmetros olhando o teste. Isso é vazamento do teste (cap. 6).
- Ler só a média da validação cruzada. Diferenças menores que o desvio entre dobras são empate.
- Achar que mais complexo é sempre melhor. Com poucos dados (como as 141 linhas do PPI-molde), modelos simples costumam ganhar.

### No seu projeto

Quantas linhas você tem? Com poucas centenas, prefira modelos de baixa capacidade (linear, logística, árvore rasa) e diga isso no artigo, citando o argumento PAC/VC: “com m pequeno, modelos de maior capacidade têm limite de generalização mais frouxo”.

**A pergunta que fica:** a regressão e a logística traçam fronteiras com somas ponderadas. Existem outros jeitos de aprender com exemplos, cada um com o seu “botão” contra o sobreajuste. Qual é o mais simples deles?

### Verifique

1. Qual a dimensão VC de um classificador linear no plano?
2. Segundo o PAC, o que acontece com o número de exemplos necessário quando o conjunto de modelos possíveis cresce?
3. Qual a relação entre regularização e dimensão VC?

*Respostas:* (1) 3. (2) Cresce, com o logaritmo de $|H|$. (3) A regularização restringe a capacidade efetiva do modelo (sua “VC efetiva”), o que reduz a folga entre o erro de treino e o erro verdadeiro.

## Prática da Unidade 2

| Onde | Capítulos | O que você vai ver |
|---|---|---|
| `AC_U1`, seções 3–4 (separar e baseline) | 5 | Chutar sempre “Adelie” acerta 44,2% do teste |
| `AC_U1`, seções 5–6 (Pipeline e escala) | 6 e 7 | k-NN sem padronizar: 80,2%; com padronização: 100% |
| `AC_U1`, seção 7 (matriz de confusão) | 9 | Onde estão os erros |
| `AC_U2`, A1–A2 (regressão, diabetes) | 5, 8 e 10 | MAE do baseline 66,9; linear, Ridge e Lasso cerca de 45 ($R^2$ = 0,48); Lasso zera 3 variáveis |
| `AC_U2`, B1–B2 (logística, câncer) | 9 | Baseline 62,7% e recall zero; logística com F1 0,962; tabela de limiares |
| `AC_U2`, B3 (curva de complexidade) | 10 e 13 | Árvore: treino chega a F1 = 1,0, validação máxima na profundidade 5 |

| Onde | Capítulos | O que você vai ver |
|---|---|---|
| Práticas de 03/11 e 10/11 | 3 e 5 a 8 | ETL da base analítica; split, dois baselines e regressão em Pipeline |

**O resultado que mais ensina:** no `AC_U1`, a única diferença entre 80% e 100% de acerto foi padronizar. Uma decisão de preparação pesou mais que a escolha do algoritmo, e só foi possível medir isso porque o teste estava separado desde o começo.

## Checklist da Unidade 2 (vale para o seu projeto)

- ☐ Separei o teste primeiro, estratificado quando é classificação.
- ☐ O baseline está na tabela de resultados.
- ☐ Toda preparação que usa estatística dos dados está dentro de um `Pipeline`.
- ☐ Padronizei antes de qualquer método baseado em distância.
- ☐ Defini a classe positiva (ou a unidade de y) e escolhi a métrica pelo custo dos erros.
- ☐ Hiperparâmetros vieram de `GridSearchCV` dentro do treino.

# Unidade 3 — Outras famílias de modelos

A Unidade 3 responde: **existem outros jeitos de aprender com exemplos?** Cada capítulo apresenta uma família como resposta a uma limitação da anterior: vizinhança (k-NN), probabilidade (Naive Bayes), perguntas legíveis (árvore), a fronteira mais segura (SVM), a transformação aprendida (redes) e, por fim, a combinação de todas (ensembles), que fecha com a **comparação justa**. Em todos, a mesma pergunta: qual é o botão que controla o sobreajuste?

## Cap. 11 — Classificadores baseados em instâncias: k-NN

**De onde viemos:** sabemos medir distâncias (Cap. 7), avaliar classificadores (Cap. 9) e controlar a complexidade (Cap. 10). O k-NN junta essas coisas da forma mais direta possível: ele não ajusta coeficiente nenhum, só guarda os exemplos.

### A ideia

O **k-NN** (*k-Nearest Neighbors*, k vizinhos mais próximos) não "aprende uma regra": ele **guarda os exemplos** de treino. Para classificar um caso novo, procura os k exemplos mais próximos e faz uma votação (na regressão, tira a média). Por isso é chamado de método **baseado em instâncias** ou **preguiçoso** (*lazy*): o trabalho fica todo para a hora de prever.

*Analogia:* estimar o aluguel justo de um apartamento olhando os 5 apartamentos mais parecidos já alugados no mesmo bairro.

### Os botões

| Hiperparâmetro | Efeito |
|---|---|
| **k** | k = 1 decora o treino (variância alta, VC enorme); k grande suaviza demais (viés alto). Escolha por validação cruzada; use k ímpar em problemas binários |
| `weights='distance'` | Vizinhos mais próximos pesam mais no voto |
| `metric` | Euclidiana (padrão), Manhattan ou outras (cap. 7) |

### Pontos fortes e fracos

- **Fortes:** simples, sem suposição sobre a forma da fronteira, explica cada decisão ("estes 5 casos parecidos eram malignos"), funciona para multiclasse naturalmente.

- **Fracos:** exige padronização (cap. 7); fica lento com bases grandes; sofre com muitas colunas (maldição da dimensionalidade); colunas irrelevantes atrapalham a distância.

### No código

```
busca = GridSearchCV(Pipeline([('esc', StandardScaler()), ('m',
KNeighborsClassifier())]),
                     {'m__n_neighbors': [3, 5, 7, 11, 15]}, cv=cv, scoring='f1')
```

No notebook, o k-NN teve F1 de 0,948 no câncer de mama, com a maior precisão entre os modelos simples (0,986), mas um recall menor (0,912). No notebook `AC_U1`, a padronização levou o mesmo k-NN de 80% para 100% de acerto nos pinguins.

### Armadilhas

- Esquecer o `StandardScaler`. É o erro mais comum com k-NN.
- Escolher k olhando o teste.

### No seu projeto

O k-NN é um ótimo segundo modelo depois da logística e dá uma explicação que o gestor entende: “municípios parecidos com este tiveram tal resultado”.

**A pergunta que fica:** o k-NN decide pela vizinhança. E se, em vez de distância, o modelo raciocinasse com **probabilidades**?

### Verifique

1. Por que o k-NN é chamado de preguiçoso?
2. O que acontece com k = 1?
3. Por que padronizar é obrigatório?

*Respostas:* (1) Porque não ajusta nada no treino; só guarda os exemplos e calcula tudo na hora de prever. (2) Ele decora o treino e sofre sobreajuste. (3) Porque a decisão depende de distâncias, dominadas pelas colunas de maior escala.

## Cap. 12 — Modelos gráficos: Naive Bayes e redes bayesianas

**De onde viemos:** a logística (Cap. 9) estima P(y | x) diretamente e o k-NN (Cap. 11) decide pela vizinhança. Os modelos gráficos fazem outro caminho: descrevem **como as variáveis dependem umas das outras** e usam probabilidade para raciocinar.

### A base: o teorema de Bayes

$$P(\text{classe} \mid \text{dados}) = \frac{P(\text{dados} \mid \text{classe}) \cdot P(\text{classe})}{P(\text{dados})}$$

- **P(classe)**, a *priori*: quão comum é a classe antes de olhar o caso.
- **P(dados | classe)**, a verossimilhança: quão típicos são esses dados para essa classe.
- **P(classe | dados)**, a *posteriori*: o que queremos, a probabilidade atualizada pela evidência.

## O que é um modelo gráfico

Um **modelo gráfico probabilístico** representa as variáveis como **nós** de um grafo e as dependências como **arestas**. Numa **rede bayesiana**, o grafo é dirigido e sem ciclos: cada seta diz "esta variável influencia diretamente aquela". Exemplo de trânsito:

*Chuva* → *Risco de acidente* ← *Horário de pico*; *Risco de acidente* → *Chamados ao SAMU*.

Com o grafo e uma tabela de probabilidades por nó, dá para responder perguntas nas duas direções. Por exemplo: "com muitos chamados ao SAMU, qual a chance de estar chovendo?". A grande vantagem é a **interpretabilidade**: o grafo pode ser desenhado com especialistas do domínio.

Outros modelos gráficos conhecidos são os modelos ocultos de Markov (HMM, para sequências, como fala e trajetórias) e os campos aleatórios de Markov (usados em segmentação de imagens).

## Naive Bayes: o modelo gráfico mais simples

O **Naive Bayes** é uma rede bayesiana em que a classe aponta para todas as variáveis, e as variáveis são supostas **independentes entre si, dada a classe**. A suposição é "ingênua" (*naive*), quase sempre falsa, mas torna o cálculo trivial:

$$P(\text{classe} \mid x_1, \ldots, x_p) \;\propto\; P(\text{classe}) \prod_{j=1}^{p} P(x_j \mid \text{classe})$$

| Variante | Para que tipo de X |
|---|---|
| `GaussianNB` | Numéricas, supondo distribuição normal em cada classe |
| `MultinomialNB` | Contagens, como frequência de palavras em textos (classificar denúncias, e-mails) |
| `BernoulliNB` | Variáveis 0/1 |

## Pontos fortes e fracos

- **Fortes:** muito rápido, funciona com poucos dados, excelente baseline para **textos**, dá probabilidades e lida bem com muitas colunas.
- **Fracos:** a suposição de independência deixa as probabilidades mal calibradas (exageradas perto de 0 ou 1). Com variáveis muito correlacionadas, perde para outros modelos.

No notebook, o `GaussianNB` teve F1 de 0,920 no câncer de mama, abaixo da logística (0,961). As 30 medidas do tumor são fortemente correlacionadas (raio, perímetro e área medem quase a mesma coisa), o que viola a suposição ingênua.

### No código

```
from sklearn.naive_bayes import GaussianNB, MultinomialNB
nb = GaussianNB().fit(X_tr, y_tr)
nb.predict_proba(X_te)[:5]
# Redes bayesianas completas: biblioteca pgmpy (fora do scikit-learn)
```

No Weka, o Naive Bayes está em *classifiers* → *bayes* → *NaiveBayes*, e as redes bayesianas em *BayesNet*.

### Armadilhas

- Ler as probabilidades do Naive Bayes como calibradas. Use-as para ordenar, não como certeza.
- Usar `GaussianNB` com contagens de palavras. O correto é o `MultinomialNB`.

### No seu projeto

Se seu projeto classifica **textos** (denúncias de ouvidoria, descrições de licitações), comece com `TfidfVectorizer` + `MultinomialNB` como baseline forte. Se o problema tem relações causais conhecidas pelos especialistas, uma rede bayesiana desenhada com eles pode valer mais que um modelo opaco.

**A pergunta que fica:** somas ponderadas, distâncias, probabilidades. Existe um modelo que qualquer gestor consiga ler, como uma lista de perguntas?

### Verifique

1. Qual é a suposição “ingênua” do Naive Bayes?
2. O que uma seta numa rede bayesiana representa?
3. Por que o Naive Bayes perdeu para a logística nos dados de câncer?

*Respostas:* (1) Que as variáveis são independentes entre si, dada a classe. (2) Uma dependência direta (influência) de uma variável sobre outra. (3) Porque as medidas são muito correlacionadas, o que viola a suposição de independência.

## Cap. 13 — Árvores de decisão

**De onde viemos:** até aqui, os modelos usam somas ponderadas (Caps. 8 e 9), distâncias (Cap. 11) ou probabilidades (Cap. 12). A árvore usa algo que qualquer gestor entende: **perguntas de sim ou não**.

## A ideia

Uma **árvore de decisão** divide os dados com perguntas sucessivas do tipo "raio médio > 15?". Cada pergunta é um **nó**, cada resposta leva a um ramo, e as **folhas** dão a previsão (a classe majoritária ou a média de y naquela folha).

*Analogia:* é o roteiro de triagem de um pronto-atendimento: "tem febre? → há quanto tempo? → tem falta de ar? → ...". A diferença é que o algoritmo escreve o roteiro sozinho a partir dos dados.

## Como a árvore escolhe as perguntas

Em cada nó, ela testa todas as colunas e todos os pontos de corte e escolhe o que deixa os grupos resultantes mais **puros**:

- **Gini** (padrão no scikit-learn): a chance de errar ao rotular um caso ao acaso pela proporção do nó. Zero = nó puro.
- **Entropia / ganho de informação** (ID3, C4.5, J48 no Weka): quanto a pergunta reduz a incerteza.
- Na regressão: a redução da variância (ou do erro quadrático) de y.

O processo é **guloso**: escolhe o melhor corte agora, sem olhar adiante, e repete até um critério de parada.

## O problema: árvores decoram

Sem limite, a árvore cresce até cada folha ter um único caso: erro de treino zero e sobreajuste máximo (cap. 10). Os freios são `max_depth`, `min_samples_leaf`, `min_samples_split` e a **poda** (`ccp_alpha`). No notebook, a curva de complexidade mostra exatamente isso no câncer de mama: com profundidade 12, o F1 no treino chega a 1,0, mas o F1 na validação cai de 0,92 (profundidade 5) para 0,89.

## Pontos fortes e fracos

- **Fortes:** interpretável (dá para desenhar e explicar); não precisa padronizar; lida com interações e relações não lineares; dá a **importância das variáveis**.
- **Fracos:** instável (variância alta: pequenas mudanças nos dados geram outra árvore); fronteiras em "escada"; sozinha, costuma perder para outros modelos. No notebook, ela ficou em último entre os modelos reais (F1 de 0,909).

A solução para a instabilidade é combinar muitas árvores (cap. 16).

## No código

```
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
arvore = DecisionTreeClassifier(max_depth=3, random_state=42).fit(X_tr, y_tr)
print(export_text(arvore, feature_names=list(X_tr.columns)))   # a "chave" em
texto
plot_tree(arvore, feature_names=X_tr.columns, filled=True)
```

### Armadilhas

- Deixar `max_depth=None` e comemorar o 100% no treino.
- Ler a importância de variável como causa. Ela só diz o quanto a coluna foi útil para os cortes.

### No seu projeto

Uma árvore rasa (profundidade 3) é uma ótima **figura para o artigo e para a banca**, mesmo que não seja o modelo final. Ela mostra ao gestor quais variáveis e limites importam.

**A pergunta que fica:** a árvore é legível, mas instável e fácil de sobreajustar. A teoria do Cap. 10 disse que controlar a capacidade melhora a generalização. Existe um modelo que procure, de propósito, a fronteira mais segura?

### Verifique

1. O que é um nó puro?
2. Por que uma árvore sem limite sobreajusta?
3. Por que não é preciso padronizar para árvores?

*Respostas:* (1) Um nó em que todos os casos têm a mesma classe (Gini zero). (2) Porque ela pode criar uma folha para cada exemplo e decorar o ruído. (3) Porque cada corte compara uma coluna com um limite, e mudar a escala da coluna só muda o valor do limite.

## Cap. 14 — Máquinas de vetores de suporte (SVM)

**De onde viemos:** a logística (Cap. 9) traça uma fronteira linear qualquer que separe bem; a teoria (Cap. 10) disse que controlar a capacidade melhora a generalização. O SVM junta as duas ideias: entre todas as fronteiras possíveis, escolhe a **mais segura**.

### A ideia: a maior margem

Imagine duas nuvens de pontos separáveis por uma reta. Infinitas retas servem. O SVM escolhe a que fica **o mais longe possível dos pontos mais próximos de cada classe**. Essa distância é a **margem**, e os pontos que a tocam são os **vetores de suporte**: só eles definem a fronteira. Os demais poderiam ser apagados sem mudar nada.

*Analogia:* traçar o limite de uma área de preservação entre duas propriedades. O traçado mais justo é o que deixa a maior faixa livre até as cercas mais próximas dos dois lados. Quem define a faixa são essas cercas, não as que estão longe.

Maximizar a margem é um jeito de **limitar a capacidade** do modelo, a ponte direta com a dimensão VC do cap. 10: uma margem larga garante boa generalização mesmo com muitas colunas.

## Margem suave: o parâmetro C

Dados reais não são perfeitamente separáveis. A **margem suave** permite alguns pontos do lado errado, com uma penalidade controlada por **C**:

- **C pequeno:** tolera mais erros e tem margem larga (mais regularização, menos sobreajuste).
- **C grande:** quase não tolera erros e tem margem estreita (risco de sobreajuste).

## O truque do kernel

E quando a fronteira é curva? O SVM leva os dados para um espaço de mais dimensões, onde eles ficam separáveis por um hiperplano, sem nunca calcular esse espaço explicitamente. Basta uma função de similaridade entre pares de pontos, o **kernel**.

| Kernel | Fronteira | Hiperparâmetros |
|---|---|---|
| `linear` | Reta / hiperplano | C |
| `rbf` (gaussiano, padrão) | Curvas flexíveis | C e `gamma` (alcance de cada ponto: gamma alto = fronteira muito recortada) |
| `poly` | Polinomial | C, grau |

## Pontos fortes e fracos

- **Fortes:** excelente com muitas colunas e poucas centenas ou milhares de linhas; fundamentação teórica sólida; robusto com bom ajuste de C e gamma.
- **Fracos: exige padronização** (usa distâncias); lento com dezenas de milhares de linhas; não dá probabilidade nativamente; pouco interpretável com kernel RBF.

No notebook, o SVM RBF com C e gamma escolhidos por `GridSearchCV` teve F1 de 0,962 e o **menor desvio entre dobras** (0,008), além da maior ROC-AUC (0,995). Foi o modelo mais estável.

## No código

```
from sklearn.svm import SVC
busca = GridSearchCV(Pipeline([('esc', StandardScaler()), ('m',
SVC(kernel='rbf'))]),
                     {'m__C': [0.1, 1, 10], 'm__gamma': ['scale', 0.01]}, cv=cv,
scoring='f1')
```

## Armadilhas

- Rodar SVM sem `StandardScaler`.
- Buscar C e gamma em grades enormes com bases grandes: o tempo explode. Comece com 3 × 3 valores em escala logarítmica.

### No seu projeto

Inclua o SVM como candidato se sua base tiver muitas colunas numéricas (indicadores, bandas espectrais). Em sensoriamento remoto, SVM e Random Forest são os classificadores clássicos de uso do solo.

**A pergunta que fica:** o kernel do SVM cria fronteiras curvas escolhendo uma transformação pronta dos dados. E se o modelo **aprendesse** sozinho essa transformação?

### Verifique

1. O que são vetores de suporte?
2. O que acontece ao aumentar muito o C?
3. Para que serve o kernel?

*Respostas:* (1) Os pontos mais próximos da fronteira, os únicos que a definem. (2) A margem estreita, o modelo tolera menos erros de treino e tende a sobreajustar. (3) Para criar fronteiras não lineares, separando os dados linearmente num espaço de mais dimensões sem calculá-lo explicitamente.

## Cap. 15 — Redes neuronais: do perceptron às redes para imagens

**De onde viemos:** a logística (Cap. 9) é uma soma ponderada passada por uma função sigmoide. Uma rede neural é, literalmente, **muitas logísticas empilhadas em camadas**, que aprendem sozinhas a transformação dos dados que o SVM (Cap. 14) precisava receber pronta.

### O neurônio artificial e o perceptron

Um **neurônio** recebe entradas $x_1 \ldots x_p$, multiplica cada uma por um **peso** $w_j$, soma um **viés** b e aplica uma **função de ativação**:

$$a = f\left(\sum_{j=1}^{p} w_j x_j + b\right)$$

O **perceptron** (Rosenblatt, 1958) usa uma ativação em degrau e aprende corrigindo os pesos a cada erro. Ele só separa classes com uma **reta** (ou um hiperplano). O exemplo clássico de falha é o XOR, que nenhuma reta separa. Essa limitação levou às redes com camadas escondidas.

### O MLP (*Multilayer Perceptron*)

- **Camada de entrada:** as colunas de X.
- **Camadas escondidas:** neurônios que combinam as entradas e criam representações intermediárias. Usam ativações **não lineares**, como a ReLU (max(0, z)), e é isso que permite fronteiras curvas.

- **Camada de saída:** sigmoide (binária), softmax (multiclasse) ou linear (regressão).

**Como aprende:** o erro na saída é propagado de trás para frente (**retropropagação**, *backpropagation*), e cada peso é ajustado um pouco na direção que reduz o erro (**descida do gradiente**). Uma passada por todos os dados é uma **época**.

*Analogia:* ajustar dezenas de registros de uma rede de irrigação. Mede-se a vazão na ponta, e cada registro é ajustado um pouco na direção que aproxima o resultado do desejado, repetidas vezes.

## Os botões

| Hiperparâmetro | Efeito |
|---|---|
| `hidden_layer_sizes` | Neurônios e camadas: (32,) = 1 camada de 32. Mais neurônios = mais capacidade (e mais risco de sobreajuste, cap. 10) |
| `alpha` | Regularização L2 dos pesos |
| `learning_rate_init`, `max_iter` | Tamanho do passo e número de épocas |
| `early_stopping=True` | Para quando a validação interna para de melhorar |

## Pontos fortes e fracos

- **Fortes:** aproxima relações muito complexas; é a base do *deep learning* (imagens, texto, voz). Imagens de câmeras de trânsito, satélite e drone são processadas com redes convolucionais, primas do MLP (veja a seção sobre imagens, a seguir).
- **Fracos:** precisa de **padronização** e de mais dados; tem muitos hiperparâmetros; é pouco interpretável; o resultado varia com a semente.

No notebook, um MLP com 32 neurônios teve F1 de 0,971 no câncer de mama, empatado na prática com a logística (0,961) quando se considera o desvio entre dobras (±0,02). Em dados tabulares de algumas centenas de linhas, isso é o esperado: a rede raramente compensa a complexidade.

## No código

```
mlp = make_pipeline(StandardScaler(),
                    MLPClassifier(hidden_layer_sizes=(32,), alpha=1e-3,
max_iter=2000,
                                  early_stopping=False, random_state=42))
```

## Redes neurais para imagens: convolução, detecção e YOLO

A ementa não traz visão computacional como item próprio, mas ela é a aplicação mais visível das redes neurais, e alguns projetos da PPI usam câmeras. Esta seção dá o mínimo para usar esses modelos **com o mesmo rigor** dos capítulos anteriores.

**Por que o MLP não basta.** Uma foto de 640 × 480 pixels coloridos tem 921.600 números. Ligar cada um a cada neurônio exige milhões de pesos, e o MLP não sabe que pixels vizinhos estão relacionados: se a placa se mover 10 pixels para o lado, para ele é outra imagem.

**A ideia da convolução.** Uma **rede convolucional** (CNN) passa pequenos **filtros** (por exemplo, 3 × 3 pixels) por toda a imagem, como uma lupa que desliza. Cada filtro aprende a reagir a um padrão: uma borda vertical, um canto, uma textura. As camadas seguintes combinam esses padrões em formas cada vez mais complexas: bordas viram caracteres, caracteres viram uma placa. Como o mesmo filtro é usado em toda a imagem, a rede reconhece o padrão em qualquer posição, com muito menos pesos.

**Três tarefas diferentes com imagens**

| Tarefa | Pergunta | Saída | Exemplo no trânsito |
|---|---|---|---|
| Classificação de imagem | O que há nesta imagem? | Uma classe | "Há congestionamento: sim/não" |
| **Detecção de objetos** | O quê e **onde**? | Caixas (x, y, largura, altura) + classe + confiança | Cada veículo e cada placa na cena |
| Reconhecimento de caracteres (OCR) | Que texto está aqui? | Sequência de caracteres | "QBA1C23" |

A detecção junta duas coisas que você já conhece: **classificação** (qual objeto) e **regressão** (as coordenadas da caixa). A leitura de placas costuma ser um **pipeline** de etapas: detectar o veículo → detectar a placa dentro dele → ler os caracteres. O erro de cada etapa se soma ao das outras, então cada uma deve ser avaliada separadamente.

**YOLO** (*You Only Look Once*) é uma família de detectores que analisa a imagem inteira de uma vez e prevê todas as caixas num só passo. Por isso é rápido o bastante para vídeo em tempo real. As versões recentes (pacote `ultralytics`) vêm **pré-treinadas** em milhões de imagens de objetos comuns.

**Aprendizagem por transferência (*transfer learning*).** Ninguém treina uma CNN do zero com algumas centenas de imagens. Parte-se de um modelo pré-treinado, que já sabe reconhecer bordas e formas, e faz-se o **ajuste fino** (*fine-tuning*) com as imagens do seu problema, já anotadas com as caixas. É o mesmo raciocínio de aproveitar o que foi aprendido antes, agora com milhões de pesos.

```
from ultralytics import YOLO
modelo = YOLO('yolov8n.pt')                          # pré-treinado: o baseline
modelo.val(data='placas.yaml')                       # mede ANTES de ajustar
modelo.train(data='placas.yaml', epochs=50, imgsz=640, seed=42)   # ajuste fino
metricas = modelo.val(data='placas.yaml', split='test')           # teste, uma vez
```

**As métricas da detecção**

- **IoU** (*Intersection over Union*): área da interseção entre a caixa prevista e a caixa verdadeira, dividida pela área da união. Vale 1 se as caixas coincidem perfeitamente e 0 se não se tocam. Uma detecção conta como acerto (VP) quando o IoU passa de um limiar, geralmente 0,5.
- Com isso, voltam as métricas do cap. 9: **precisão** (das caixas previstas, quantas eram placas de verdade?) e **recall** (das placas que existiam, quantas foram encontradas?).
- **mAP@0,5** (*mean Average Precision*): a área sob a curva precisão × recall, com média entre as classes. É o número que os artigos de detecção reportam. O **mAP@0,5:0,95** é mais exigente: faz a média para limiares de IoU de 0,5 a 0,95.
- Na leitura (OCR): **acerto da placa inteira** (todos os caracteres certos) e **acerto por caractere**. Para o gestor, só a placa inteira importa: um caractere errado é outro veículo.

**O protocolo da apostila vale igual**

1. **Split por câmera ou por dia, nunca por frame.** Frames seguidos de um vídeo são quase idênticos. Um split aleatório põe "cópias" do treino no teste e infla o mAP (vazamento, caps. 5 e 6).
2. **Baseline:** o modelo pré-treinado sem ajuste fino, avaliado no mesmo conjunto de teste.
3. **Teste anotado pelo grupo**, separado antes de tudo e usado uma vez.
4. **Análise de erros por condição:** dia × noite, chuva, ângulo da câmera, moto × carro, placa antiga × Mercosul. Um mAP alto no geral pode esconder um desempenho péssimo à noite (o mesmo alerta de justiça do cap. 24).
5. **LGPD:** placa é dado pessoal, porque identifica o proprietário. Defina a finalidade, guarde o mínimo e, se possível, use só agregados (contagens por horário e tipo de veículo).

## Armadilhas

- Não padronizar: o treino fica lento e instável.
- Ignorar o aviso `ConvergenceWarning`. Ele diz que faltaram épocas; aumente `max_iter`.
- Escolher rede neural "porque é IA". Na banca, a pergunta será: ela superou a logística? Por quanto, e com que custo de interpretação?

## No seu projeto

Use o MLP como um dos 2–4 candidatos se tiver mais de mil linhas ou relações claramente não lineares. Para imagens de câmeras, use detecção com aprendizagem por transferência (YOLO) e avalie com o protocolo da seção acima: split por câmera, baseline pré-treinado, mAP, precisão e recall.

**A pergunta que fica:** cada família tem os seus erros: a árvore é instável, a logística é rígida, o k-NN é sensível a colunas inúteis, a rede pede muito dado. E se, em vez de escolher uma, juntássemos várias?

## Verifique

1. Por que um perceptron não resolve o XOR?

2. O que a função de ativação não linear permite?
3. O que é retropropagação?

*Respostas:* (1) Porque ele só cria fronteiras lineares, e o XOR não é linearmente separável. (2) Fronteiras curvas; sem ela, várias camadas equivalem a uma só. (3) O cálculo de quanto cada peso contribuiu para o erro, propagando o erro da saída para as camadas anteriores, para ajustar os pesos por descida do gradiente.

# Cap. 16 — Combinação de classificadores (ensembles)

**De onde viemos:** conhecemos as principais famílias (Caps. 8, 9 e 11 a 15), cada uma com os seus erros: a árvore é instável, a logística é rígida, o k-NN é sensível a colunas inúteis. **Combinar** modelos diferentes cancela parte desses erros. Este capítulo fecha a parte supervisionada com a comparação justa entre todos.

## A ideia: a sabedoria das multidões

Se vários modelos razoáveis erram em casos **diferentes**, a votação entre eles erra menos que cada um sozinho. A palavra-chave é **diversidade**: dez cópias do mesmo modelo não ajudam.

*Analogia:* a média das notas de vários avaliadores independentes de uma banca é mais confiável que a nota de um avaliador só, desde que eles não combinem as notas entre si.

## As quatro estratégias

| Estratégia | Como gera diversidade | Combina como | Ataca | Exemplos |
|---|---|---|---|---|
| **Bagging** | Cada modelo treina numa amostra *bootstrap* (sorteio com reposição) | Votação ou média | Variância | `BaggingClassifier` |
| **Random Forest** | Bagging de árvores + sorteio de colunas em cada corte | Votação | Variância | `RandomForestClassifier` |
| **Boosting** | Modelos em sequência, cada um focando nos erros do anterior | Soma ponderada | Viés (e variância) | `AdaBoost`, `GradientBoosting`, XGBoost, LightGBM |

| Estratégia | Como gera diversidade | Combina como | Ataca | Exemplos |
|---|---|---|---|---|
| **Votação / Stacking** | Famílias diferentes de modelos | Voto (`VotingClassifier`) ou um modelo que aprende a combiná-los (`StackingClassifier`) | Ambos | Logística + MLP + RF |

- **Votação dura:** cada modelo vota numa classe. **Votação suave:** média das probabilidades; costuma ser melhor, mas exige `predict_proba`.
- **Random Forest** é o melhor "primeiro modelo forte" em dados tabulares: pouco ajuste, não precisa padronizar e dá a importância das variáveis. O erro **OOB** (*out-of-bag*, nos exemplos que ficaram fora de cada amostra) dá uma validação "de graça".
- **Gradient Boosting** costuma vencer competições com dados tabulares, mas tem mais hiperparâmetros (número de árvores, taxa de aprendizado, profundidade) e sobreajusta se mal ajustado.

## O que o notebook mostrou

| Modelo (câncer de mama, validação cruzada) | F1 médio | Desvio |
|---|---|---|
| Votação suave (logística + MLP + RF) | 0,971 | 0,021 |
| Logística | 0,961 | 0,022 |
| Random Forest | 0,949 | 0,026 |
| Gradient Boosting (padrão) | 0,934 | 0,023 |
| Árvore única (melhor profundidade) | 0,909 | 0,031 |

A Random Forest supera a árvore única em 4 pontos de F1: é o bagging reduzindo a variância. A votação ficou no topo, mas a diferença para a logística sozinha (0,01) é menor que o desvio entre dobras. Pelo critério de parcimônia, a logística foi a escolhida no notebook.

## No código

```
from sklearn.ensemble import RandomForestClassifier, VotingClassifier,
StackingClassifier
rf = RandomForestClassifier(n_estimators=300, oob_score=True,
random_state=42).fit(X_tr, y_tr)
print(rf.oob_score_, rf.feature_importances_)
voto = VotingClassifier([('lr', logit), ('mlp', mlp), ('rf', rf)],
```

```
voting='soft')
pilha = StackingClassifier([('lr', logit), ('rf', rf)],
final_estimator=LogisticRegression())
```

### Armadilhas

- Combinar modelos muito parecidos. Sem diversidade, não há ganho.
- Usar ensemble para ganhar 0,005 e perder toda a interpretabilidade. Na banca, pese o ganho contra o custo de explicar.
- Stacking com vazamento: o modelo final deve aprender com previsões fora da dobra (o `StackingClassifier` já faz isso).

### No seu projeto

Inclua uma Random Forest entre os candidatos: é o teste de "existe algo não linear que os modelos simples não pegaram?". Se ela não superar a logística com folga, fique com a logística e diga isso no artigo.

**A pergunta que fica:** tudo até aqui dependeu de um y conhecido. E quando não existe rótulo? O primeiro passo é enxergar dados de muitas colunas: dá para resumi-las em poucas?

### Verifique

1. Por que diversidade é essencial num ensemble?
2. Bagging reduz viés ou variância?
3. Qual a diferença entre votação e stacking?

*Respostas:* (1) Porque só há cancelamento de erros se os modelos erram em casos diferentes. (2) Principalmente variância. (3) A votação combina por uma regra fixa (voto ou média); o stacking treina um modelo para aprender a melhor combinação.

## Prática da Unidade 3

| Onde | Capítulos | Resultado de referência |
|---|---|---|
| `AC_U2`, B4 (comparar 9 modelos + baseline) | 11 a 16 | Votação 0,971, MLP 0,971, SVM 0,962, logística 0,961 … árvore 0,909 (F1) |
| `AC_U2`, B5 (teste, uma vez) | 5 | Logística: F1 0,951, recall 0,925, precisão 0,980 |
| Prática de 17/11 | 5, 10, 11, 13 e 16 | Validação cruzada, árvores, sobreajuste e GridSearchCV |

**O resultado que mais ensina:** os quatro primeiros modelos estão dentro do desvio entre dobras (cerca de ±0,02) uns dos outros. Na prática, empataram. A escolha da logística vem da parcimônia e da interpretabilidade, e não da terceira casa decimal. É o tipo de raciocínio que a banca espera ouvir.

## Checklist da Unidade 3 (vale para o seu projeto)

- ☐ Comparei pelo menos 2 famílias de modelos com o mesmo `cv` e a mesma métrica.
- ☐ Sei dizer qual é o botão contra o sobreajuste de cada modelo que usei.
- ☐ Li média **e** desvio entre dobras antes de declarar um vencedor.
- ☐ Em caso de empate, escolhi o modelo mais simples e explicável.
- ☐ Abri o teste uma única vez e discuti o limiar de decisão.

# Unidade 4 — Sem y, e até a decisão

A Unidade 4 responde: **o que fazer quando não existe y, e como levar qualquer resultado até uma decisão?** Muitos problemas públicos não têm rótulo: não existe "município do tipo A", mas o gestor quer saber que tipos existem. A unidade começa resumindo colunas (PCA), passa por quatro maneiras de um ponto pertencer a um grupo (por inteiro, com probabilidade, com grau, num mapa), procura o que não pertence a grupo nenhum (anomalias) e fecha com as aplicações.

## Cap. 17 — Redução de dimensionalidade: PCA

**De onde viemos:** na parte supervisionada sempre houve um y. Agora não há. Vimos que muitas colunas atrapalham as distâncias (Cap. 7) e que algumas colunas são redundantes (correlação de 0,87 entre nadadeira e massa, Cap. 4). Dá para resumir várias colunas em poucas sem perder o essencial?

### A ideia

O **PCA** (*Principal Component Analysis*, análise de componentes principais) cria **novas colunas**, chamadas componentes, que são combinações das originais:

- O **1º componente (PC1)** é a direção em que os dados mais variam.
- O **2º (PC2)** é a direção de maior variação que sobra, **perpendicular** ao primeiro (sem correlação com ele).
- E assim por diante, até o número de colunas originais.

Se os 2 primeiros componentes guardam, digamos, 88% da variância total, podemos olhar os dados num gráfico 2D quase sem perder informação.

*Analogia:* fotografar um carro. A foto (2D) perde uma dimensão, mas, se você escolher o ângulo certo, ainda reconhece o modelo, o tamanho e a cor. O PCA escolhe automaticamente "o melhor ângulo": aquele que mostra a maior parte da variação.

### Como funciona (o mínimo)

1. Padronizar as colunas. Sem isso, a coluna de maior unidade vira o PC1 sozinha.
2. Calcular a matriz de covariância e seus **autovetores** (as direções, que são os componentes) e **autovalores** (quanto de variância cada direção explica).
3. Ordenar pelos autovalores e manter os k primeiros.

A **variância explicada** de cada componente é seu autovalor dividido pela soma de todos. Uma regra prática é manter componentes até somar 80% a 95%.

### No código

```
from sklearn.decomposition import PCA
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

pca = make_pipeline(StandardScaler(), PCA(n_components=2))
Z = pca.fit_transform(X_treino_num)                 # 2 colunas novas
print(pca[-1].explained_variance_ratio_)           # fração da variância em cada
componente
print(pca[-1].components_)                         # pesos (loadings) de cada
coluna original
```

Os **loadings** dizem o que cada componente "significa". Se o PC1 tem pesos altos em nadadeira, massa e comprimento do bico, ele é um eixo de "tamanho do pinguim".

### Para que usar

- **Visualizar** dados com muitas colunas em 2D (o uso mais comum na EDA).
- **Reduzir ruído e redundância** antes de um modelo sensível a distâncias (k-NN, K-Means).
- **Comprimir** muitas variáveis correlacionadas, como bandas espectrais de uma imagem de satélite. Em sensoriamento remoto, o PCA é clássico para resumir dezenas de bandas.

### Armadilhas

- Ajustar o PCA na base toda antes do split. Ele é pré-processamento e vai para dentro do `Pipeline` (Cap. 6).
- Achar que o PCA "escolhe as melhores variáveis para prever y". Ele **não olha o y**. Maximiza variância, que pode não ter nada a ver com o alvo.
- Perder a interpretabilidade: "PC3" não diz nada ao gestor público. Se a explicação importa, prefira selecionar variáveis originais.

### No seu projeto

Se sua base tem muitas colunas parecidas (indicadores socioeconômicos, por exemplo), faça um gráfico PC1 × PC2 colorido pelo y ou pelo grupo. É uma das melhores figuras de EDA para o artigo.

**A pergunta que fica:** no gráfico PC1 × PC2 os pontos formaram nuvens separadas. Como transformar essas nuvens em **grupos**, e o que faz um grupo ser bom?

### Verifique

1. Por que padronizar antes do PCA?
2. O PCA usa o y?
3. O que significa "PC1 explica 69% da variância"?

*Respostas:* (1) para que nenhuma coluna domine só pela unidade de medida. (2) Não, é não supervisionado. (3) Que a projeção dos dados nessa única direção preserva 69% da variabilidade total das colunas padronizadas.

# Cap. 18 — Análise de agrupamento: o que é um bom grupo

**De onde viemos:** sabemos medir semelhança (Cap. 7), padronizar (Cap. 6) e reduzir dimensão (Cap. 17). Agrupar é usar tudo isso para **descobrir** grupos que ninguém rotulou.

## A ideia

**Agrupamento** (*clustering*) divide os exemplos em grupos com duas propriedades:

- **Coesão:** exemplos do mesmo grupo são parecidos entre si.
- **Separação:** exemplos de grupos diferentes são diferentes.

Não existe "o agrupamento correto". Existem agrupamentos **úteis** para um propósito. Os mesmos municípios podem ser agrupados por perfil socioeconômico (para política social) ou por perfil de uso do solo (para política ambiental).

*Analogia:* uma rede de supermercados que separa os clientes em perfis ("compra no fim do mês", "compra pouco e sempre", "só em promoção") para fazer uma oferta diferente a cada um. O agrupamento encontra esses perfis automaticamente, com muitas variáveis ao mesmo tempo.

## As famílias de métodos (a ementa pede quatro)

| Família | Cada exemplo pertence a... | Método representativo | Cap. |
|---|---|---|---|
| **Partição** | exatamente um grupo | K-Means, K-Medoids | 20 |
| (Hierárquico e densidade) | um grupo, numa árvore ou por região densa | Ward, DBSCAN | 20 (contraste) |
| **Probabilístico** | cada grupo, com uma probabilidade | Mistura de gaussianas (GMM) | 21 |
| **Difuso** | cada grupo, com um grau de pertinência | Fuzzy C-Means | 22 |
| **Mapa auto-organizável** | um neurônio de uma grade 2D que preserva vizinhanças | SOM (Kohonen) | 23 |

## Como avaliar sem rótulo

| Critério | O que mede | Leitura |
|---|---|---|
| **Inércia** (WCSS) | Soma das distâncias² de cada ponto ao centro do seu grupo | Sempre cai com k; procure o **cotovelo** |
| **Silhueta** | Para cada ponto: (b − a) / max(a, b), sendo a a distância média ao próprio grupo e b ao grupo vizinho mais próximo | −1 a 1; acima de 0,5 é estrutura forte, 0,25–0,5 é razoável |
| **Davies-Bouldin** | Razão entre dispersão interna e separação dos centros | Menor é melhor |
| **BIC** (probabilístico) | Ajuste do modelo penalizado pelo número de parâmetros | Menor é melhor (cap. 20) |
| **ARI** (só se houver rótulo) | Concordância entre duas partições, corrigida pelo acaso | 1 = idênticas; 0 = acaso. Serve para checar, não para escolher |

No notebook, com 178 vinhos e 13 medidas, cotovelo, silhueta (0,285) e Davies-Bouldin apontaram k = 3. Quando os critérios concordam, a escolha é fácil. Quando discordam, decide a utilidade: quantos grupos o gestor consegue tratar de forma diferente?

## Armadilhas

- Agrupar sem padronizar (cap. 7).
- Tratar os grupos como verdades. Eles são uma **descrição** útil, não classes naturais.
- Escolher k só pela inércia. Ela sempre melhora com mais grupos.

## No seu projeto

Se o seu projeto não tem y, a pergunta analítica vira: “que perfis de ___ existem, e que ação diferente cada perfil pede?”. O PPI-molde usa agrupamento como complemento para descrever os municípios atendidos.

**A pergunta que fica:** sabemos julgar um agrupamento. Qual é o jeito mais direto de produzir um?

## Verifique

1. Por que a inércia sozinha não serve para escolher k?
2. O que significa uma silhueta perto de zero?
3. Para que serve o ARI se o agrupamento é não supervisionado?

*Respostas:* (1) Porque ela sempre diminui com mais grupos; com k = n ela vale zero. (2) Que o ponto está na fronteira entre dois grupos. (3) Para checar o método em dados que têm rótulo, ou para comparar duas partições entre si.

# Cap. 19 — Agrupamento por partição: K-Means e K-Medoids

**De onde viemos:** sabemos o que é um bom grupo (Cap. 18). O K-Means é o jeito mais direto de buscar grupos coesos.

## A ideia: K-Means

Você escolhe k. O algoritmo repete dois passos até estabilizar:

1. **Atribuir:** cada ponto vai para o centro (**centróide**) mais próximo.
2. **Atualizar:** cada centróide vira a média dos pontos do seu grupo.

Cada rodada nunca aumenta a inércia, então o algoritmo sempre converge, mas pode parar num ótimo local. Por isso rodamos várias vezes com inícios diferentes (`n_init=10`) e ficamos com a melhor. O início `k-means++` espalha os centros iniciais.

*Analogia:* instalar k bases do SAMU numa cidade. Cada bairro é atendido pela base mais próxima; depois cada base é movida para o centro dos bairros que atende; repete-se até nenhum bairro mudar de base.

## Limitações e o K-Medoids

- Supõe grupos “redondos” e de tamanhos parecidos. Grupos alongados ou em forma de anel saem errados.
- É sensível a extremos, porque a média é puxada por eles.
- Exige k definido antes.

O **K-Medoids** (PAM) troca o centróide (uma média, que pode não existir nos dados) por um **medoide**: o exemplo real mais central do grupo. É mais robusto a extremos, aceita qualquer distância (inclusive para dados categóricos) e cada grupo fica representado por um caso concreto (“o município típico deste grupo é X”). Ele está no pacote `scikit-learn-extra` (`KMedoids`).

## Dois contrastes úteis

- **Hierárquico aglomerativo:** começa com cada ponto sozinho e vai juntando os grupos mais próximos. O resultado é um **dendrograma**, que mostra todas as escalas de agrupamento de uma vez; você “corta” na altura desejada. O método de **Ward** junta os grupos que menos aumentam a variância interna.
- **DBSCAN:** grupos são regiões **densas**; pontos isolados viram **ruído** (rótulo −1). Não precisa de k, acha formas irregulares e já detecta anomalias. Funciona muito bem com coordenadas

geográficas, como endereços de acidentes de trânsito para achar os pontos críticos da cidade. Sofre em muitas dimensões.

## O que o notebook mostrou

Com os 178 vinhos (13 medidas padronizadas), o K-Means com k = 3 formou grupos de 65, 51 e 62 vinhos. O dendrograma de Ward mostrou três ramos bem separados. O DBSCAN, em 13 dimensões, achou 2 grupos e marcou 42 vinhos como ruído, um exemplo prático da maldição da dimensionalidade.

## No código

```
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN
Z = StandardScaler().fit_transform(X)
km = KMeans(n_clusters=3, n_init=10, random_state=42).fit(Z)
ward = AgglomerativeClustering(n_clusters=3, linkage='ward').fit_predict(Z)
db = DBSCAN(eps=0.5, min_samples=5).fit_predict(coordenadas_em_km)   # -1 =
ruído
```

## Armadilhas

- Esquecer o `n_init` e confiar numa única inicialização.
- Usar K-Means em dados com muitas variáveis categóricas. Prefira K-Medoids com uma distância adequada (ou K-Prototypes).
- Escolher o `eps` do DBSCAN “no chute”. Use o gráfico das distâncias ao k-ésimo vizinho ordenadas e procure o joelho.

## No seu projeto

O K-Means é o primeiro método a testar. Se seus dados forem pontos no mapa (ocorrências, focos, equipamentos públicos), teste também o DBSCAN.

**A pergunta que fica:** o K-Means põe cada ponto num grupo e pronto. Um ponto na fronteira entre dois grupos não merece uma resposta mais honesta?

## Verifique

1. Quais são os dois passos do K-Means?
2. Qual a vantagem do medoide sobre o centróide?
3. Quando o DBSCAN é preferível ao K-Means?

*Respostas:* (1) Atribuir cada ponto ao centro mais próximo e recalcular os centros como médias. (2) É um exemplo real e é robusto a extremos. (3) Com grupos de forma irregular, presença de ruído e poucas dimensões, como coordenadas geográficas.

# Cap. 20 — Agrupamento probabilístico: misturas gaussianas

**De onde viemos:** o K-Means (Cap. 19) dá a cada ponto um grupo e pronto. Mas um vinho, ou município, na fronteira entre dois grupos merece uma resposta mais honesta: “70% de chance de ser do grupo A”. É o que o modelo probabilístico faz.

## A ideia

Supõe-se que os dados foram **gerados por uma mistura** de k distribuições (geralmente gaussianas), cada uma com sua média, seu espalhamento (covariância) e seu peso na mistura. Agrupar é descobrir essas distribuições e calcular, para cada ponto, a **probabilidade de ter vindo de cada uma**.

*Analogia:* a altura dos alunos de uma escola que reúne turmas do fundamental e do médio forma dois “morros” sobrepostos. Um aluno de 1,55 m, no vale entre eles, pode ser de qualquer turma; o modelo diz com que probabilidade é de cada uma.

## O algoritmo EM

O ajuste usa o algoritmo **EM** (*Expectation–Maximization*), um primo “suave” do K-Means:

1. **E (expectativa):** com as gaussianas atuais, calcula a probabilidade de cada ponto pertencer a cada uma (a “responsabilidade”).
2. **M (maximização):** reestima médias, covariâncias e pesos usando as responsabilidades como pesos.
3. Repete até a verossimilhança parar de subir.

Se as covariâncias forem iguais e esféricas e cada ponto for atribuído “com 100%”, o EM vira exatamente o K-Means.

## Escolhendo k e a forma dos grupos

- **BIC** (*Bayesian Information Criterion*): mede o ajuste penalizando o número de parâmetros. Menor é melhor. É a mesma ideia de regularização do cap. 10, agora na escolha do número de grupos.
- `covariance_type`: `'spherical'` (redondos), `'diag'` (elípticos alinhados aos eixos), `'full'` (elípticos em qualquer direção, com mais parâmetros e mais dados necessários).

## O que o notebook mostrou

Com covariância diagonal, o BIC foi mínimo em k = 3 (5.557, contra 5.984 com k = 2), confirmando a escolha da silhueta. Apenas 5 dos 178 vinhos tiveram pertinência máxima abaixo de 0,9. O caso mais incerto ficou com 58% num grupo e 42% em outro. Esses são os casos que pedem um olhar humano.

### No código

```
from sklearn.mixture import GaussianMixture
bics = {k: GaussianMixture(k, covariance_type='diag',
random_state=42).fit(Z).bic(Z) for k in range(1, 7)}
gmm = GaussianMixture(3, covariance_type='diag', random_state=42).fit(Z)
proba = gmm.predict_proba(Z)          # uma coluna por grupo, linhas somam 1
incertos = proba.max(axis=1) < 0.9
```

O modelo também dá a **densidade** de cada ponto (`score_samples`): pontos com densidade muito baixa são anomalias (cap. 23).

### Armadilhas

- Usar `'full'` com poucos dados e muitas colunas: são parâmetros demais e o ajuste fica instável.
- Ler a probabilidade como certeza externa. Ela vale **dentro** da suposição de que os dados são uma mistura de gaussianas.

### No seu projeto

Se a decisão pública depende do grupo (por exemplo, a linha de crédito de cada perfil de produtor), use a probabilidade para criar uma faixa de "revisão manual" para os casos de fronteira.

**A pergunta que fica:** a mistura gaussiana supõe que os dados vieram de "sinos". Dá para pertencer um pouco a cada grupo sem supor distribuição nenhuma?

### Verifique

1. O que o passo E calcula?
2. Como o BIC se relaciona com a regularização?
3. Que vantagem o GMM tem sobre o K-Means?

*Respostas:* (1) A probabilidade de cada ponto pertencer a cada componente. (2) Ambos penalizam a complexidade para evitar sobreajuste. (3) Dá probabilidades de pertinência e aceita grupos elípticos e de tamanhos diferentes.

## Cap. 21 — Agrupamento difuso: Fuzzy C-Means

**De onde viemos:** o modelo probabilístico (Cap. 20) deu a cada ponto uma probabilidade por grupo. O agrupamento **difuso** (*fuzzy*) chega a algo parecido por outro caminho, o da lógica difusa, sem supor nenhuma distribuição.

## A ideia: pertencer “um pouco”

Na lógica clássica, um elemento pertence ou não a um conjunto (0 ou 1). Na **lógica difusa** (Zadeh, 1965), ele pertence com um **grau** entre 0 e 1. Um bairro de **transição** entre área residencial e comercial é 0,6 residencial e 0,4 comercial, e isso não é incerteza sobre um rótulo verdadeiro: a própria área é de transição.

No **Fuzzy C-Means** (FCM, Bezdek, 1981), cada ponto i tem um grau de pertinência $\mu_{ij}$ em cada grupo j, com os graus de um ponto somando 1.

## Como funciona

O FCM alterna dois passos, como o K-Means, mas com pesos:

1. **Centros:** cada centro é a média de **todos** os pontos, ponderada por $\mu_{ij}$ elevado a m.
2. **Pertinências:** cada ponto recebe pertinência maior nos centros mais próximos:

$$\mu_{ij} = \frac{1}{\sum_{k=1}^{c} \left( \frac{\|x_i - c_j\|}{\|x_i - c_k\|} \right)^{\frac{2}{m-1}}}$$

O **fuzzificador** m > 1 controla o quanto as fronteiras são borradas. Com m → 1, o FCM vira o K-Means; com m grande, todos os pontos ficam igualmente em todos os grupos. O valor usual é m = 2.

## Como avaliar

O **coeficiente de partição** (média de $\mu^2$) vai de 1/c (totalmente difuso) a 1 (partição nítida). No notebook, com 3 grupos, ele deu 0,476: bem acima do mínimo de 0,333, mas longe de 1. Os vinhos têm estrutura, com bastante sobreposição. Atribuindo cada vinho ao grupo de maior pertinência, o FCM reproduziu exatamente a partição do K-Means (ARI = 1,0 entre os dois).

## Probabilístico × difuso

| | Mistura gaussiana (cap. 20) | Fuzzy C-Means |
|---|---|---|
| Número por grupo | Probabilidade | Grau de pertinência |
| Pergunta | “Qual componente gerou este ponto?” | “O quanto este ponto é parecido com cada protótipo?” |
| Suposição | Dados vêm de distribuições gaussianas | Nenhuma sobre distribuição; só distância |
| Escolha de k | BIC | Coeficiente de partição, silhueta difusa |

### No código

O scikit-learn não tem FCM. O notebook implementa o algoritmo em cerca de 15 linhas de NumPy, uma boa oportunidade de ver o algoritmo por dentro. Bibliotecas prontas: `scikit-fuzzy` (`skfuzzy.cluster.cmeans`) e `fcmeans`.

```
U, centros = fuzzy_c_means(Z, c=3, m=2.0)   # U: uma linha por ponto, uma coluna
por grupo
grupo = U.argmax(axis=1)                    # "desfuzzificar" quando a decisão
precisar de um rótulo
```

### Armadilhas

- "Desfuzzificar" cedo demais e jogar fora justamente a informação de transição.
- Esquecer que o FCM herda as limitações do K-Means: grupos esféricos e sensibilidade à escala.

### No seu projeto

O difuso combina com fenômenos de transição: classes de uso do solo, níveis de vulnerabilidade social, regiões de influência de serviços públicos. Mostrar no dashboard "município 60% perfil A, 40% perfil B" pode ser mais honesto que um rótulo único.

**A pergunta que fica:** até aqui os grupos são listas de pontos. E se quiséssemos um **mapa** em que grupos parecidos ficam vizinhos?

### Verifique

1. O que o fuzzificador m controla?
2. O que significa um coeficiente de partição igual a 1/c?
3. Por que a pertinência difusa não é uma probabilidade?

*Respostas:* (1) O quanto as fronteiras são borradas. (2) Que todos os pontos pertencem igualmente a todos os grupos: não há estrutura. (3) Porque mede grau de semelhança a um protótipo, não a chance de um evento, e não supõe um modelo gerador.

## Cap. 22 — Mapas auto-organizáveis (SOM)

**De onde viemos:** vimos redes neurais supervisionadas (Cap. 15), o PCA para enxergar dados em 2D (Cap. 17) e quatro maneiras de pertencer a um grupo (Caps. 18 a 21). O **SOM** (*Self-Organizing Map*, Kohonen, 1982) é uma rede neural **não supervisionada** que agrupa e desenha um mapa ao mesmo tempo.

### A ideia

O SOM é uma **grade 2D de neurônios** (por exemplo, 6 × 6). Cada neurônio guarda um vetor de pesos do mesmo tamanho que um exemplo, como se fosse um "exemplo protótipo". O treino:

1. Sorteia um exemplo.

2. Encontra o neurônio cujos pesos são mais parecidos com ele: o **vencedor** (BMU, *best matching unit*).
3. Move o vencedor **e seus vizinhos na grade** um pouco na direção do exemplo. Os vizinhos se movem menos, conforme a distância na grade.
4. Repete milhares de vezes, diminuindo aos poucos a taxa de aprendizado e o raio da vizinhança.

Como vizinhos na grade aprendem juntos, **exemplos parecidos acabam em neurônios vizinhos**. A grade vira um mapa que preserva a topologia dos dados: regiões próximas no mapa são parecidas nos dados.

*Analogia:* uma rede de pesca jogada sobre um relevo de dados. Cada nó é puxado para os pontos mais próximos, mas os nós estão amarrados aos vizinhos, então a rede inteira se molda ao relevo sem rasgar.

## Como ler um SOM

- **Mapa de ocupação (hit map):** quantos exemplos caíram em cada neurônio.
- **Matriz-U:** para cada neurônio, a distância média aos vizinhos. Regiões claras (distância pequena) são grupos; faixas escuras (distância grande) são **fronteiras**.
- **Planos de componentes:** o valor de uma variável em cada neurônio. Mostram onde no mapa cada variável é alta.

## SOM × K-Means × PCA

| | K-Means | PCA | SOM |
|---|---|---|---|
| Saída | k grupos | Eixos lineares | Grade 2D de protótipos |
| Preserva vizinhança entre grupos? | Não | Em parte (projeção linear) | Sim, inclusive para relações não lineares |
| Uso típico | Segmentar | Resumir e visualizar | Visualizar e explorar dados complexos; depois agrupar os neurônios |

No notebook, um SOM 6 × 6 treinado com 3.000 iterações nos vinhos separou os três grupos do K-Means em regiões contíguas do mapa, com uma faixa escura na matriz-U entre elas. Dois métodos muito diferentes chegaram à mesma estrutura, o que aumenta a confiança nela.

## No código

O scikit-learn não tem SOM. O notebook implementa um em cerca de 15 linhas de NumPy. Para projetos, o pacote `minisom` é simples (`pip install minisom`):

```
from minisom import MiniSom
som = MiniSom(6, 6, Z.shape[1], sigma=1.5, learning_rate=0.5, random_seed=42)
som.train_random(Z, 3000)
```

```
umatriz = som.distance_map()     # matriz-U
posicao = som.winner(Z[0])       # neurônio vencedor do primeiro exemplo
```

### Armadilhas

- Não padronizar (o SOM usa distâncias).
- Grade grande demais para poucos dados: muitos neurônios vazios. Uma regra comum é cerca de 5·√n neurônios no total.
- Tratar cada neurônio como um grupo. Os grupos são **regiões** do mapa, delimitadas pela matriz-U (ou por um K-Means aplicado aos neurônios).

### No seu projeto

O SOM é uma ótima figura de exploração para muitos indicadores (saúde, educação, renda por município): municípios vizinhos no mapa têm perfis parecidos, mesmo estando longe geograficamente.

**A pergunta que fica:** sabemos achar o que é **típico**: grupos, regiões do mapa, direções principais. E o caso que não se parece com nada?

### Verifique

1. O que é o neurônio vencedor?
2. Por que os vizinhos do vencedor também são atualizados?
3. O que uma faixa escura na matriz-U indica?

*Respostas:* (1) O neurônio cujos pesos são mais parecidos com o exemplo apresentado. (2) Para que neurônios vizinhos aprendam padrões parecidos, o que preserva a vizinhança no mapa. (3) Uma fronteira: neurônios vizinhos com protótipos muito diferentes.

## Cap. 23 — Detecção de anomalias

**De onde viemos:** sabemos medir dispersão (Cap. 4), distância (Cap. 7), projetar os dados em poucas dimensões (Cap. 17) e achar o que é típico (Caps. 18 a 22). Juntando tudo, podemos procurar o caso que **foge do padrão**.

### A ideia

Uma **anomalia** (*outlier*, caso atípico) é um exemplo muito diferente da maioria. Ela pode ser:

- um **erro**: uma idade de 290 anos digitada no lugar de 29, um sensor com defeito;
- um **caso real e raro que interessa**: um contrato público com preço dez vezes acima do normal, um consumo de energia de um prédio público que dobrou de um mês para o outro.

A mesma técnica serve aos dois propósitos: **limpar** dados (antes de modelar) e **produzir alertas** para auditoria ou fiscalização (Cap. 24). Anomalia não é o mesmo que ruído: o ruído é variação aleatória sem significado.

## Do mais simples ao mais sofisticado

| Método | Ideia | Quando usar |
|---|---|---|
| **Regra do IQR** (boxplot) | Anômalo se < Q1 − 1,5·IQR ou > Q3 + 1,5·IQR | Uma variável por vez; robusto e explicável |
| **Escore z** | Anômalo se \|valor − média\| / desvio > 3 | Uma variável aproximadamente simétrica; sensível aos próprios extremos |
| **Isolation Forest** | Árvores que cortam os dados ao acaso isolam um ponto estranho com poucos cortes | Várias variáveis, bases grandes; é o ponto de partida recomendado |
| **LOF** (*Local Outlier Factor*) | Compara a densidade em volta do ponto com a dos vizinhos (usa distâncias, Cap. 7) | Quando o "normal" varia de região para região |
| **Erro de reconstrução do PCA** | Pontos mal representados pelos k primeiros componentes são suspeitos | Muitas variáveis correlacionadas |

Um caso pode ser normal em cada variável isolada e anômalo na **combinação**: um pinguim com bico de Gentoo e massa de Adelie. Só os métodos multivariados (Isolation Forest, LOF, PCA) percebem isso.

## No código

```
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor

X_esc = StandardScaler().fit_transform(X_num)
iso = IsolationForest(contamination=0.02, random_state=42).fit(X_esc)
escore = -iso.score_samples(X_esc)          # quanto maior, mais anômalo
suspeitos = iso.predict(X_esc) == -1        # -1 marca anomalia

lof = LocalOutlierFactor(n_neighbors=20, contamination=0.02)
suspeitos_lof = lof.fit_predict(X_esc) == -1
```

O parâmetro `contamination` é a fração esperada de anomalias. Ele não é descoberto pelo método: é uma decisão sua, ligada à capacidade de quem vai verificar os alertas.

### Armadilhas

- **Apagar anomalias automaticamente.** Primeiro investigue: se for erro, corrija ou remova; se for real, ela pode ser justamente o que o projeto procura.
- Remover atípicos **olhando o teste**. A regra de limpeza deve ser definida com o treino.
- Tratar o escore de anomalia como prova. Ele é um **indício** que prioriza a verificação humana.

### No seu projeto

Rode um Isolation Forest na base analítica e olhe os 10 casos mais estranhos. Para cada um, classifique como erro de dado, caso real interessante ou falso alarme. Muitas GovTechs nascem exatamente dessa lista: “quais contratos, obras ou unidades merecem uma auditoria primeiro?”.

**A pergunta que fica:** temos todas as ferramentas: prever com rótulo, agrupar sem rótulo, achar o estranho. Falta a pergunta da apresentação: isso ajuda alguém a decidir melhor?

### Verifique

1. Qual a diferença entre anomalia e ruído?
2. Por que um caso pode ser normal em cada coluna e anômalo no conjunto?
3. Quem define o `contamination`?

*Respostas:* (1) anomalia é um caso raro e significativo; ruído é variação aleatória sem significado. (2) Porque a combinação de valores é rara, mesmo que cada valor sozinho seja comum. (3) A equipe, com base no problema e na capacidade de verificar os alertas.

## Cap. 24 — Aplicações: da previsão à decisão

**De onde viemos:** temos as ferramentas: preparar dados, prever com rótulo, agrupar sem rótulo, detectar anomalias. Este capítulo fecha a escada com a pergunta da apresentação: **isso ajuda alguém a decidir melhor?**

### Os quatro padrões de aplicação no setor público

| Padrão | Técnica | Saída para o gestor | Exemplo GovTech |
|---|---|---|---|
| **Prever e priorizar** | Regressão ou classificação + ranking | Lista ordenada de onde agir primeiro | Municípios com maior abstenção prevista; obras com maior risco de atraso |
| **Prever e alocar** | Previsão + otimização | Carteira ou rota ótima | Onde investir R$ 1 |

| Padrão | Técnica | Saída para o gestor | Exemplo GovTech |
|---|---|---|---|
| | com restrição | sob orçamento | milhão em mobilização (PPI-molde) |
| **Segmentar e tratar** | Agrupamento + perfis | Tipologia com ação por tipo | Perfis de escolas para programas diferentes |
| **Detectar e verificar** | Anomalias | Fila de casos para auditoria humana | Contratos com preço atípico; focos fora da época |

## O elo que mais falha: da previsão à decisão

No PPI-molde, o modelo prevê a abstenção com erro médio de 1,23 pontos percentuais no teste (contra 1,96 de repetir a eleição anterior e 2,13 de prever a média). Uma otimização escolhe onde investir. A primeira versão do molde, com um modelo bem mais fraco (erro de 2,14), ensinou uma lição importante: **a otimização amplifica o erro da previsão**. Ela concentrou cerca de 30% do orçamento num município em que o modelo superestimou a abstenção, e uma regra simples se saiu melhor. Com a previsão melhor, a otimização voltou a vencer. Três lições:

1. Use previsões **fora da amostra** (`cross_val_predict`) para decidir, nunca previsões no próprio treino.
2. Compare a decisão do modelo com uma **regra simples**, como você compara o modelo com o baseline.
3. Leve a **incerteza** para a decisão: cenário conservador, faixa de revisão manual, probabilidades (cap. 20) ou pertinências (cap. 21).

**Veja o projeto inteiro funcionando:** o notebook `PPI_Molde_Comparece_MT` percorre todas as etapas com dados do TSE (141 municípios de MT), do ETL ao dashboard, e cada seção indica o capítulo da apostila correspondente. Problema e decisão (caps. 1–3) → dados e checagens (caps. 3 e 4 e 6) → split e baseline (cap. 5) → comparação de modelos (caps. 8–16) → perfis por K-Means (caps. 18 e 19) → previsões fora da amostra e otimização sob orçamento → CSV para Power BI e dashboard web. Na versão `INICIAL`, você completa 6 lacunas. As práticas semanais constroem o mesmo projeto, um pedaço por semana.

## Ética, LGPD e justiça

- **Dados pessoais:** prefira dados agregados (por município, escola) quando a decisão é coletiva. Se precisar de dados individuais, minimize, anonimize e justifique (LGPD).
- **Viés:** um modelo treinado com decisões passadas aprende os vieses dessas decisões. Verifique o desempenho **por grupo** (região, porte do município): um recall de 95% no geral pode esconder 70% num grupo.

- **Transparência:** o cidadão afetado tem direito de entender o critério. Modelos interpretáveis (logística, árvore rasa) e listas de variáveis importantes ajudam.
- **Humano no circuito:** o modelo prioriza e sugere; quem decide e responde pela decisão é uma pessoa.

### Comunicar: dashboard e artigo

- **Dashboard:** uma tela por pergunta do gestor. Cada gráfico deve apoiar uma ação; mostre o erro do modelo ao lado da previsão.
- **Artigo SBC, parte de AC:** (1) problema, X e y; (2) dados e preparo, com a regra anti-vazamento; (3) protocolo: split, validação cruzada, métrica e por quê; (4) tabela baseline × modelos com média e desvio; (5) teste final; (6) limitações e ética. A teoria do cap. 10 ajuda a justificar a escolha de modelos simples com poucos dados.

### No seu projeto

Escolha um dos quatro padrões e escreva a frase: *"o modelo de AC produz ___, que o gestor usa para ___, e sabemos que funciona porque ___"*. Se a última lacuna não tiver um número comparado com um baseline, ainda falta evidência.

### Verifique

1. Por que usar previsões fora da amostra para decidir?
2. Como um modelo pode ser injusto mesmo com boa métrica geral?
3. Qual o papel da regra simples na avaliação de uma decisão?

*Respostas:* (1) Porque previsões no treino são otimistas e levariam a decisões baseadas em erros subestimados. (2) Se o desempenho for ruim num subgrupo, compensado pelos demais. (3) É o baseline da decisão: mostra se a inteligência do modelo agrega algo em relação ao bom senso.

## Prática da Unidade 4

| Onde | Capítulos | Resultado de referência |
|---|---|---|
| `AC_U1`, seção 8 (PCA) | 17 | 2 componentes guardam 88% da variância, e as espécies se separam no plano |
| `AC_U3`, seção 2 (avaliação) | 18 | Cotovelo, silhueta (0,285) e Davies-Bouldin apontam k = 3 |

| Onde | Capítulos | Resultado de referência |
|---|---|---|
| `AC_U3`, seção 3 (K-Means, Ward, DBSCAN) | 19 | Grupos de 65, 51 e 62; DBSCAN marca 42 vinhos como ruído |
| `AC_U3`, seção 4 (mistura gaussiana) | 20 | BIC mínimo em k = 3; 5 vinhos de fronteira (pertinência < 0,9) |
| `AC_U3`, seção 5 (Fuzzy C-Means) | 21 | Coeficiente de partição 0,476 (mínimo 0,333) |
| `AC_U3`, seção 6 (SOM 6 × 6) | 22 | Grupos em regiões contíguas; fronteiras na matriz-U |
| `AC_U1`, seção 9 (anomalias) | 23 | IQR por coluna × Isolation Forest multivariado; os 6 casos mais estranhos |
| `AC_U3`, seção 7 (checagem e perfis) e `PPI_Molde_Comparece_MT` | 18 e 24 | ARI contra as cultivares: K-Means 0,897; o projeto completo, do CSV ao dashboard |
| Prática de 24/11 | 7, 17, 18, 19 e 24 | PCA, K-Means e a carteira sob orçamento (mochila) |

**O resultado que mais ensina:** sem nunca ver o rótulo, o K-Means recuperou quase exatamente as três cultivares (ARI de 0,897). A tabela de perfil em unidades originais traduz os grupos em algo que um enólogo entende. É esse passo, do número ao nome, que transforma um agrupamento em decisão.

## Checklist da Unidade 4 (vale para o seu projeto)

- ☐ Padronizei antes de reduzir dimensão ou agrupar.
- ☐ Fiz um gráfico PC1 × PC2 antes de agrupar.
- ☐ Escolhi o número de grupos com pelo menos 2 critérios e o bom senso do domínio.
- ☐ Identifiquei os casos de fronteira (probabilidade ou pertinência) e as anomalias, e decidi o que fazer com elas.
- ☐ Descrevi cada grupo em unidades originais e dei nomes compreensíveis.
- ☐ Escrevi a frase “o modelo produz ___, que o gestor usa para ___” (Cap. 24).

# Apêndices

## A. Guia rápido: que modelo e que métrica?

**Passo 1 — Que tarefa?**

| Tenho y? | y é... | Tarefa | Comece por | Depois teste |
|---|---|---|---|---|
| Sim | Quantidade | Regressão | `DummyRegressor` → `LinearRegression/Ridge` | `RandomForestRegressor`, `GradientBoostingRegressor` |
| Sim | Categoria | Classificação | `DummyClassifier` → `LogisticRegression` | k-NN, árvore rasa, `RandomForestClassifier`, SVM |
| Sim | Texto → categoria | Classificação de texto | `TfidfVectorizer` + `MultinomialNB` | `LogisticRegression` sobre TF-IDF |
| Não | — | Agrupamento | `KMeans` + silhueta | `GaussianMixture`, Ward, DBSCAN (geográfico) |
| Não | — | Anomalias | Regra do IQR | `IsolationForest`, `LocalOutlierFactor` |

**Passo 2 — Que métrica?**

| Situação | Métrica principal |
|---|---|
| Regressão para gestor | MAE (com $R^2$ de apoio) |
| Erros grandes são muito piores | RMSE |
| Classes equilibradas, erros com custo parecido | Acurácia + F1 |
| Perder um positivo é grave | Recall (com precisão mínima) |
| Cada alarme custa caro | Precisão |

| Situação | Métrica principal |
|---|---|
| Positivo raro | PR-AUC e F1; nunca só acurácia |
| Agrupamento | Silhueta, Davies-Bouldin, BIC |

**Passo 3 — O protocolo (sempre o mesmo)**

```
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, stratify=y,
random_state=42)
cv = StratifiedKFold(5, shuffle=True, random_state=42)           # KFold na
regressão
pipe = Pipeline([('prep', preparo), ('modelo',
LogisticRegression(max_iter=5000))])
busca = GridSearchCV(pipe, {'modelo__C': [0.1, 1, 10]}, cv=cv,
scoring='f1').fit(X_tr, y_tr)
print(busca.best_score_)                                         # escolha pela
validação
print(busca.score(X_te, y_te))                                   # teste: uma
vez, no fim
```

**Quem precisa de padronização:** k-NN, SVM, MLP, regressões regularizadas, K-Means, PCA, SOM, Fuzzy C-Means. **Quem não precisa:** árvores, Random Forest e Gradient Boosting.

## B. Weka em uma página

O Weka (Universidade de Waikato) faz, com cliques, o mesmo protocolo do scikit-learn. Ele foi usado no miniestudo da turma e é ótimo para testar ideias rápido.

| Conceito | No Weka (*Explorer*) | No scikit-learn |
|---|---|---|
| Carregar dados | *Preprocess* → *Open file* (.arff ou .csv) | `pd.read_csv` |
| Escolher o y | Última coluna, ou *(Nom) class* na aba *Classify* | `y = df['coluna']` |
| Baseline | `rules → ZeroR` | `DummyClassifier` |
| Naive Bayes | `bayes → NaiveBayes` | `GaussianNB` |
| Rede bayesiana | `bayes → BayesNet` | (pgmpy) |
| Logística | `functions → Logistic` | `LogisticRegression` |
| k-NN | `lazy → IBk` | `KNeighborsClassifier` |
| Árvore | `trees → J48` (C4.5) | `DecisionTreeClassifier(criterion='entropy')` |

| Conceito | No Weka (*Explorer*) | No scikit-learn |
|---|---|---|
| Random Forest | `trees → RandomForest` | `RandomForestClassifier` |
| SVM | `functions → SMO` | `SVC` |
| MLP | `functions → MultilayerPerceptron` | `MLPClassifier` |
| Validação cruzada | *Test options* → *Cross-validation*, *Folds 10* | `cross_validate(cv=10)` |
| Agrupamento | Aba *Cluster*: `SimpleKMeans`, `EM`, `HierarchicalClusterer` | `KMeans`, `GaussianMixture`, `AgglomerativeClustering` |
| Comparar com teste estatístico | *Experimenter* | (manual) |

**Atenção:** os filtros de *Preprocess* (como `Normalize`) são aplicados na base inteira, antes da validação: é vazamento (cap. 6). Para evitar, use `meta → FilteredClassifier`, que aplica o filtro dentro de cada dobra, como o `Pipeline`.

## C. Glossário português–inglês

| Português | Inglês | Cap. |
|---|---|---|
| Agrupamento | Clustering | 19 |
| Alvo, rótulo | Target, label | 2 |
| Anomalia, caso atípico | Anomaly, outlier | 8 |
| Aprendizagem por reforço | Reinforcement learning | 2 |
| Atributo, variável preditora | Feature | 2 |
| Árvore de decisão | Decision tree | 16 |
| Baseline, linha de base | Baseline | 9 |
| Classe positiva | Positive class | 12 |
| Combinação de classificadores | Ensemble | 18 |
| Componente principal | Principal component | 7 |
| Detecção de objetos | Object detection | 15 |

| Português | Inglês | Cap. |
|---|---|---|
| Dimensão VC | VC dimension | 10 |
| Grau de pertinência | Membership degree | 22 |
| Hiperparâmetro | Hyperparameter | 10 |
| Interseção sobre união | Intersection over Union (IoU) | 15 |
| Limiar | Threshold | 12 |
| Mapa auto-organizável | Self-organizing map (SOM) | 23 |
| Margem | Margin | 17 |
| Matriz de confusão | Confusion matrix | 12 |
| Mistura de gaussianas | Gaussian mixture model (GMM) | 21 |
| Padronização | Standardization, scaling | 5 |
| Precisão / revocação | Precision / recall | 12 |
| Regularização | Regularization | 10 |
| Rede convolucional | Convolutional neural network (CNN) | 15 |
| Rede neuronal (neural) | Neural network | 15 |
| Resíduo | Residual | 11 |
| Retropropagação | Backpropagation | 15 |
| Sobreajuste / subajuste | Overfitting / underfitting | 10 |
| Transferência de aprendizado | Transfer learning | 15 |
| Treino / validação / teste | Train / validation / test | 9 |
| Validação cruzada | Cross-validation | 10 |
| Vazamento de dados | Data leakage | 5 |
| Vetores de suporte | Support vectors | 17 |
| Viés e variância | Bias and variance | 10 |

# D. Leituras recomendadas

**Livros**

- Géron, A. *Mãos à Obra: Aprendizado de Máquina com Scikit-Learn, Keras & TensorFlow*. Alta Books. A referência prática mais próxima desta apostila (cap. 1–9 do livro).
- Faceli, K.; Lorena, A. C.; Gama, J.; Almeida, T. A.; Carvalho, A. C. P. L. F. *Inteligência Artificial: uma abordagem de Aprendizado de Máquina*. 2. ed. LTC, 2021. Em português e alinhado à ementa: instâncias, probabilísticos, SVM, redes, agrupamento e teoria.
- Mitchell, T. *Machine Learning*. McGraw-Hill, 1997. Cap. 17 para aprendizagem PAC e dimensão VC.
- Haykin, S. *Redes Neurais: princípios e prática*. Bookman. Perceptron, MLP e mapas auto-organizáveis.
- Grus, J. *Data Science do Zero*. Alta Books. Implementa os algoritmos do zero em Python.

**On-line e gratuitos**

- Guia do usuário do scikit-learn: um exemplo para cada modelo desta apostila.
- Machine Learning Crash Course (Google): vídeos curtos e exercícios.
- Portal de Dados Abertos do Governo Federal, IBGE/SIDRA, Dados Abertos do TSE: fontes para a PPI.

**Material da disciplina**

- Notebooks `AC_U1`, `AC_U2` e `AC_U3` (versão INICIAL; o mapa capítulo → seção está na Prática de cada unidade), na pasta `apostila/notebooks` do repositório da disciplina.
- `PPI_Molde_Comparece_MT`: um projeto completo, do CSV ao dashboard, para usar como modelo (cap. 24).

# E. Práticas semanais

Toda prática semanal fica numa pasta própria em `atividades/` do repositório, com a data e o tema no nome: um `LEIAME` (pergunta, objetivo, critérios e saída esperada), um notebook `INICIAL` com lacunas `TODO` e, no fim do notebook, uma célula de verificação que mostra ✅ ou ❌ para cada item.

| Data | Pasta em atividades/ | Prática | Capítulos |
|---|---|---|---|
| 27/10 | `2026-10-27_Aula01_O_Problema` | O problema: onde o eleitor de MT deixa de votar? (pergunta, unidade, y, X e a ficha do problema) | 1, 2, 3 e 4 |
| 03/11 | `2026-11-03_Aula02_ETL_Base_Analiti` | Da fonte à base analítica: o ETL do PPI-molde (TSE → base_analitica.csv, | 3, 4, 6 e 7 |

| Data | Pasta em atividades/ | Prática | Capítulos |
|---|---|---|---|
| | `ca` | dicionário e EDA) | |
| 10/11 | `2026-11-10_Aula03_Experimento_Honesto` | Experimento honesto: split, dois baselines e regressão linear em Pipeline | 5, 6 e 8 |
| 17/11 | `2026-11-17_Aula04_Validacao_Cruzada_Arvores` | Qual modelo escolher? Validação cruzada, árvores, sobreajuste e GridSearchCV | 5, 10, 11, 13 e 16 |
| 24/11 | `2026-11-24_Aula05_Perfis_e_Decisao` | Da previsão à decisão: PCA, K-Means e a carteira sob orçamento (mochila) | 7, 17, 18, 19 e 24 |
| 01/12 | `2026-12-01_Aula06_Tribunal_do_Pipeline` | Tribunal do Pipeline: auditar e consertar um experimento (ensaio da banca) | 2, 5, 6, 9, 11 e 24 |

**Práticas extras** podem ser incluídas a qualquer semana, sempre neste mesmo formato. Quando sair uma nova, baixe o repositório de novo (ou rode `git pull`), depois de salvar os seus notebooks em outro lugar.